

บทที่ 3 ชนิดข้อมูล (Data Types)

ชนิดข้อมูล คือ สิ่งที่ใช้กำหนดลักษณะและขอบเขตของข้อมูลนั้น ๆ โดยข้อมูลที่มีชนิดของข้อมูลแตกต่างกัน ก็จะเก็บข้อมูลได้ในลักษณะที่ต่างกัน และขอบเขตของข้อมูลที่เก็บได้ก็จะไม่เท่ากันด้วย

ในภาษาไพธอนมีชนิดข้อมูลพื้นฐานที่ถูกสร้างมาในตัว (Built-in Data Types) หลากหลายประเภท ดังต่อไปนี้

3.1 ชนิดข้อมูลตัวเลข (Numeric Types)

ใช้สำหรับจัดเก็บค่าตัวเลขเพื่อการคำนวณทางคณิตศาสตร์ แบ่งออกเป็น 3 ประเภท คือ

➤ int (Integer - จำนวนเต็ม)

ใช้สำหรับจัดเก็บจำนวนเต็ม ซึ่งสามารถเป็นได้ทั้งจำนวนเต็มบวก จำนวนเต็มลบ หรือศูนย์ จะเป็นเลขฐานสิบ เลขฐานสอง (นำหน้าด้วย 0b) เลขฐานแปด (นำหน้าด้วย 0o) เลขฐานสิบหก (นำหน้าด้วย 0x) ก็ได้

ตัวอย่างจำนวนเต็มบวก

```
positive_number = 12345
year_born = 2000
print(f"จำนวนเต็มบวก: {positive_number}, ปีเกิด: {year_born}")
```

ตัวอย่างจำนวนเต็มลบ

```
negative_number = -987
temperature_celsius = -5
print(f"จำนวนเต็มลบ: {negative_number}, อุณหภูมิ: {temperature_celsius}")
```

ตัวอย่างค่าศูนย์

```
zero_value = 0
balance_start = 0
print(f"ค่าศูนย์: {zero_value}, ยอดเริ่มต้น: {balance_start}")
```

ตัวอย่างเลขฐานสิบ

```
decimal_number = 42
large_decimal = 1_000_000 # สามารถใช้ underscore เพื่อให้อ่านง่ายขึ้น
print(f"เลขฐานสิบ: {decimal_number}, เลขเยอะ ๆ: {large_decimal}")
```

ตัวอย่างเลขฐานสอง

```
binary_num = 0b1010 # เทียบเท่ากับ 10 ในฐานสิบ
binary_flag = 0B1111 # เทียบเท่ากับ 15 ในฐานสิบ
print(f"เลขฐานสอง 0b1010: {binary_num}, เลขฐานสอง 0b1111: {binary_flag}")
```

ตัวอย่างเลขฐานแปด

```
octal_num = 0o12 # เทียบเท่ากับ 10 ในฐานสิบ
octal_permission = 0O755 # เทียบเท่ากับ 493 ในฐานสิบ
print(f"เลขฐานแปด 0o12: {octal_num}, เลขฐานแปด 0o755: {octal_permission}")
```

ตัวอย่างเลขฐานสิบหก

```
hex_num = 0xA # เทียบเท่ากับ 10 ในฐานสิบ
rgb_color_value = 0xFF # เทียบเท่ากับ 255 ในฐานสิบ (มักใช้ในรหัสสี)
memory_address = 0x1A2B # เทียบเท่ากับ 6700 ในฐานสิบ
print(f"เลขฐานสิบหก 0xA: {hex_num}, เลขฐานสิบหก 0xFF: {rgb_color_value}, เลขฐานสิบหก 0x1A2B: {memory_address}")
```

ตัวอย่างการใช้ฟังก์ชัน bin()

```
decimal_number_1 = 10
binary_representation_1 = bin(decimal_number_1)
print(f"ค่าทศนิยม {decimal_number_1} ในฐานสองคือ: {binary_representation_1}")
```

ตัวอย่างการใช้ฟังก์ชัน oct()

```
octal_representation_1 = oct(decimal_number_1)
print(f"ค่าทศนิยม {decimal_number_1} ในฐานแปดคือ: {octal_representation_1}")
```

ตัวอย่างการใช้ฟังก์ชัน hex()

```
hex_representation_1 = hex(decimal_number_1)
print(f"ค่าทศนิยม {decimal_number_1} ในฐานสิบหกคือ: {hex_representation_1}")
```

➤ float (Floating-Point Number - จำนวนจริง/ทศนิยม)

ใช้สำหรับจัดเก็บค่า จำนวนจริง (Real Numbers) ซึ่งรวมถึงจำนวนที่มีส่วนเป็นทศนิยม หรือจำนวนที่มีขนาดใหญ่หรือเล็กมากที่แสดงในรูปสัญกรณ์วิทยาศาสตร์ (Scientific Notation) float เป็นชนิดข้อมูลพื้นฐานที่สำคัญสำหรับการคำนวณที่ต้องการความแม่นยำในระดับทศนิยม

ตัวอย่างจำนวนจริงบวกและลบ

```
pi_value = 3.14159
temperature_celsius = -15.5
zero_point_five = 0.5
print(f"ค่า Pi: {pi_value}, อุณหภูมิ: {temperature_celsius}, ครึ่งหนึ่ง: {zero_point_five}")
```

สำหรับจำนวนจริงที่มีขนาดใหญ่มากหรือเล็กมาก สามารถใช้สัญกรณ์วิทยาศาสตร์ โดยใช้ตัวอักษร e (หรือ E) แทน คูณด้วยสิบยกกำลัง

ตัวอย่าง

```
large_number_e = 3.58e2 # เทียบเท่ากับ 3.58 * (10 ยกกำลัง 2) = 3.58 * 100 = 358.0
small_number_e = 1.23e-4 # เทียบเท่ากับ 1.23 * (10 ยกกำลัง -4) = 1.23 * 0.0001 = 0.000123
gigahertz_value = 5.0e9 # 5.0 พันล้าน (5,000,000,000.0)

print(f"3.58e2 มีค่าเท่ากับ: {large_number_e}")
print(f"1.23e-4 มีค่าเท่ากับ: {small_number_e}")
print(f"5.0e9 มีค่าเท่ากับ: {gigahertz_value}")
```

ข้อควรทราบเกี่ยวกับความแม่นยำของ float

เนื่องจาก float ถูกจัดเก็บในรูปแบบฐานสอง การแทนค่าทศนิยมบางค่าที่ในระบบฐานสิบสามารถแสดงได้ถูกต้อง (เช่น 0.1) อาจไม่สามารถแทนได้อย่างแม่นยำในรูปแบบฐานสอง ทำให้เกิดความคลาดเคลื่อนเล็กน้อยในการคำนวณ

ตัวอย่าง

```
result = 0.1 + 0.2
print(result) # ผลลัพธ์อาจเป็น 0.30000000000000004 แทนที่จะเป็น 0.3
```

➤ complex (Complex Number - จำนวนเชิงซ้อน)

ใช้สำหรับจัดเก็บจำนวนเชิงซ้อน (Complex Numbers) ที่ประกอบด้วยส่วนจริง (real part) และส่วนจินตภาพ (imaginary part) ซึ่งเขียนในรูปแบบ $a+bj$

ตัวอย่าง

```
complex_number1 = 3 + 2j      # ส่วนจริงคือ 3, ส่วนจินตภาพคือ 2
complex_number2 = -1.5 - 0.5j # ส่วนจริงคือ -1.5, ส่วนจินตภาพคือ -0.5
complex_number3 = 5j          # ส่วนจริงเป็น 0 (ไม่ระบุ), ส่วนจินตภาพคือ 5

print(f"complex_number1: {complex_number1}, ชนิด: {type(complex_number1)}")
print(f"complex_number2: {complex_number2}")
print(f"complex_number3: {complex_number3}")
```

ตัวอย่างการใช้ complex()

```
x = 3 + 4j
y = complex(2, -5) # ฟังก์ชันที่ใช้สำหรับสร้างจำนวนเชิงซ้อน
z = x + y
print(z) # ผลลัพธ์: (5-1j)
```

คุณสมบัติ (Attributes) และเมธอด (Methods) ที่สำคัญของ complex Object

- **.real:** แอดทริบิวต์สำหรับเข้าถึงส่วนจริงของจำนวนเชิงซ้อน (เป็นชนิด float)
- **.imag:** แอดทริบิวต์สำหรับเข้าถึงส่วนจินตภาพของจำนวนเชิงซ้อน (เป็นชนิด float)
- **.conjugate():** เมธอดที่ส่งคืนค่าสังยุค (Conjugate) ของจำนวนเชิงซ้อน (คือการเปลี่ยนเครื่องหมายของส่วนจินตภาพ)

ตัวอย่าง

```
z = 3 + 2j
print(f"ส่วนจริงของ {z}: {z.real}")      # ผลลัพธ์: 3.0
print(f"ส่วนจินตภาพของ {z}: {z.imag}")  # ผลลัพธ์: 2.0
print(f"สังยุคของ {z}: {z.conjugate()}")  # ผลลัพธ์: (3-2j)
```

3.2 ชนิดข้อมูลตรรกะ (Boolean)

ชนิดข้อมูลตรรกะ (Boolean Type) หรือ bool ในภาษา Python เป็นชนิดข้อมูลพื้นฐานที่ใช้สำหรับจัดเก็บ ค่าความจริง (Truth Values) ซึ่งมีเพียงสองสถานะเท่านั้น คือ True (จริง) และ False (เท็จ) ชนิดข้อมูลนี้มีความสำคัญอย่างยิ่งในการควบคุมการไหลของโปรแกรม (Control Flow) และการตัดสินใจเชิงตรรกะ

หมายเหตุ ตัวอักษรตัวแรกของ True และ False จะต้องเป็นตัวพิมพ์ใหญ่เสมอ

ตัวอย่าง กำหนดค่า True หรือ False ให้กับตัวแปรได้โดยตรง

```
is_active = True
has_errors = False
print(f"สถานะการทำงาน: {is_active}")
print(f"มีข้อผิดพลาดหรือไม่: {has_errors}")
```

ตัวอย่าง การใช้ตัวดำเนินการเปรียบเทียบจะส่งคืนค่า bool เสมอ

```
num1 = 10
num2 = 20
is_equal = (num1 == num2)      # 10 เท่ากับ 20 หรือไม่? -> False
is_greater = (num2 > num1)      # 20 มากกว่า 10 หรือไม่? -> True
print(f"num1 เท่ากับ num2: {is_equal}")
print(f"num2 มากกว่า num1: {is_greater}")
```

ตัวอย่าง การใช้งานในคำสั่งเงื่อนไข เพื่อควบคุมการไหลของโปรแกรม

```
user_logged_in = True
if user_logged_in:
    print("ยินดีต้อนรับเข้าสู่ระบบ")
else:
    print("กรุณาเข้าสู่ระบบ")
```

การแปลงค่าเป็น Boolean (Boolean Conversion)

ใน Python ค่าเกือบทุกชนิดสามารถถูกประเมินเป็นค่าตรรกะได้ โดยใช้ฟังก์ชัน bool()

- ค่าที่ถือว่าเป็น False
 - None

- 0 (สำหรับ int และ float)
- '' (String ว่างเปล่า)
- [] (List ว่างเปล่า)
- () (Tuple ว่างเปล่า)
- {} (Dictionary ว่างเปล่า)
- set() (เซตว่างเปล่า)
- ค่าอื่น ๆ ทั้งหมด (ที่ไม่ใช่ False) จะถือว่าเป็น True

ตัวอย่าง

```
print(f"bool(0): {bool(0)}")      # ผลลัพธ์: False
print(f"bool(1): {bool(1)}")      # ผลลัพธ์: True
print(f"bool(''): {bool('')}")    # ผลลัพธ์: False
print(f"bool('hello'): {bool('hello')}") # ผลลัพธ์: True
print(f"bool([]): {bool([])}")     # ผลลัพธ์: False
print(f"bool([1, 2]): {bool([1, 2])}") # ผลลัพธ์: True
```

3.3 ชนิดข้อมูลสายอักขระ (String)

ชนิดข้อมูลสายอักขระ หรือ String (ในภาษาไพธอนใช้ชื่อย่อว่า str) คือชนิดข้อมูลที่ใช้สำหรับจัดเก็บ ลำดับของตัวอักขระ (Sequence of Characters) โดยตัวอักขระแต่ละตัวอาจเป็นตัวอักษร ตัวเลข สัญลักษณ์ หรือช่องว่าง String ไม่สามารถเปลี่ยนแปลงได้ (Immutable) เมื่อมีการเปลี่ยนแปลงค่าจะเกิดการสร้างอ็อบเจกต์ใหม่ขึ้นแทน

อักขระแต่ละตัวใน String จะมีตำแหน่ง (Index) โดย Index เริ่มต้นที่ 0 สำหรับตัวอักขระแรก และ Index ติดลบจากท้ายสตริง (-1 คือตัวสุดท้าย) ดังตัวอย่างด้านล่าง

C	o	m	S	c	i		R	U
---	---	---	---	---	---	--	---	---

ค่า Index แบบบวก 0 1 2 3 4 5 6 7 8

ค่า Index แบบลบ -9 -8 -7 -6 -5 -4 -3 -2 -1

ลักษณะสำคัญของ String ในไพธอน

- เป็นลำดับของอักขระ (Sequence of Characters)

- สร้าง String ได้โดยการใช้ัญประกาศเดี่ยว (' ') ัญประกาศคู่ (" ") หรือัญประกาศสามชั้น (' ' ' ' หรือ " " " ") ล้อมรอบข้อความที่ต้องการ
- ไม่สามารถแก้ไขค่าโดยตรง (Immutable)
- รองรับ Unicode สามารถเก็บอักขระได้หลากหลายภาษา
- สามารถเข้าถึงอักขระแต่ละตัวได้ด้วยการใช้ดัชนี (indexing) และสามารถตัดข้อความบางส่วน (slicing) ได้

ตัวอย่างการสร้าง String โดยใช้ single quote (' ')

ใช้สำหรับสร้างสตริงแบบบรรทัดเดียว นิยมใช้ตามความสะดวกหรือเมื่อสตริงมี ัญประกาศอีกชนิดอยู่ภายใน

ตัวอย่าง

```
single_quote_string = 'Hello, Python!'
double_quote_string = "My name is Yoke."
quote_in_string = 'He said, "Welcome!"'
print(single_quote_string)
print(double_quote_string)
print(quote_in_string)
```

ตัวอย่างการสร้าง String โดยใช้ Triple double quotes (" " " ") และ triple single quotes (' ' ' ')

ใช้สำหรับสร้างสตริงแบบหลายบรรทัด (Multi-line Strings) โดยไม่ต้องใช้ตัวอักษรพิเศษ \n หรือสตริงที่ต้องการมี ัญประกาศเดี่ยว/คู่จำนวนอยู่ภายใน (ไม่ต้อง escape \) และนิยมใช้สำหรับเขียน docstrings คือข้อความอธิบายฟังก์ชัน คลาส หรือโมดูล

ตัวอย่าง

```
multi_line_string = """
This is a string
that spans
multiple lines.
"""

doc_string_example = """
Function to demonstrate multi-line string.
```

It can include 'single' and "double" quotes easily.

'''

print(multi_line_string)

print(doc_string_example)

การดำเนินการพื้นฐานและเมธอดของสตริง

การเข้าถึงตัวอักขระด้วยดัชนี (Indexing)

การเข้าถึงอักขระในสตริงทำได้โดยระบุ Index ของสตริง โดยใช้ [] ครอบ Index

รูปแบบการใช้งาน

string[index]

string_variable[index]

Index เริ่มต้นที่ 0 สำหรับตัวอักขระแรก และสามารถใช้ Index ติดลบจากท้ายสตริงได้ (-1 คือตัวสุดท้าย)

ตัวอย่าง

```
word = "PYTHON"
```

```
print(f"ตัวอักขระตัวแรก: {word[0]}") # ผลลัพธ์: P
```

```
print(f"ตัวอักขระตัวที่สาม: {word[2]}") # ผลลัพธ์: T
```

```
print(f"ตัวอักขระตัวสุดท้าย: {word[-1]}") # ผลลัพธ์: N
```

การแบ่งส่วนสตริง (Slicing)

Slicing เป็นการเข้าถึงบางส่วนของสตริง โดยระบุช่วงที่ต้องการดึงออกมา

รูปแบบการใช้งาน

string[start:end:step]

string_variable[start:end:step]

โดยที่

- start: ตำแหน่งเริ่มต้น (index) ที่ต้องการดึงข้อมูล (ถ้าไม่ระบุจะเริ่มที่ 0)
- end: ตำแหน่งสุดท้ายที่ต้องการดึงข้อมูล (แต่ไม่รวมตำแหน่งนี้)
- step: ระยะห่างระหว่างแต่ละค่าที่ถูกเลือก (ถ้าไม่ระบุจะเป็น 1)

ตัวอย่าง

```
text = "Python Programming"
print(text[0:6])    # ผลลัพธ์: Python
print(text[7:])     # ผลลัพธ์: Programming
print(text[:6])     # ผลลัพธ์: Python
print(text[-6:])    # ผลลัพธ์: amming
print(text[::-2])   # ผลลัพธ์: Pto rgamn
```

การเชื่อมสตริง (Concatenation)

ใช้เครื่องหมาย + เพื่อรวมสตริงสองตัวขึ้นไปเข้าด้วยกัน

ตัวอย่าง

```
s1 = "Hello"
s2 = "World"
result = s1 + " " + s2
print(result) # ผลลัพธ์: Hello World
```

การทำซ้ำสตริง (Repetition)

ใช้เครื่องหมาย * เพื่อทำซ้ำสตริงตามจำนวนครั้งที่กำหนด

ตัวอย่าง

```
s = "Hi! "
print(s * 3) # ผลลัพธ์: Hi! Hi! Hi!
```

การหาความยาวของสตริง

ใช้ฟังก์ชัน len() เพื่อหาจำนวนตัวอักษรในสตริง

ตัวอย่าง

```
s = "Hello"
print(len(s)) # ผลลัพธ์: 5
```

เมธอดของสตริง (String Methods)

เมธอดของสตริง คือ ฟังก์ชันที่ถูกกำหนดไว้ในคลาส str ของภาษาไพธอน ซึ่งช่วยให้สามารถประมวลผลและจัดการกับข้อมูลสายอักขระ (string) ได้อย่างสะดวกและมีประสิทธิภาพ

เมธอดเหล่านี้จะถูกเรียกใช้ผ่านตัวแปรสตริงโดยใช้เครื่องหมายจุด (.) ตามด้วยชื่อเมธอดและวงเล็บ
เมธอดส่วนใหญ่จะ ไม่เปลี่ยนแปลงสตริงเดิม แต่จะคืนค่าสตริงใหม่ที่ผ่านการประมวลผลแล้ว

- lower()
 - แปลงสตริงเป็นตัวพิมพ์เล็กทั้งหมด

ตัวอย่าง

```
"Hello".lower() # ผลลัพธ์: 'hello'
```

- upper()
 - แปลงสตริงเป็นตัวพิมพ์ใหญ่ทั้งหมด

ตัวอย่าง

```
"Hello".upper() # ผลลัพธ์: 'HELLO'
```

- strip()
 - ลบช่องว่าง (whitespace) ที่อยู่หัวและท้ายสตริง

ตัวอย่าง

```
" hello ".strip() # ผลลัพธ์: 'hello'
```

- split(separator)
 - แบ่งสตริงออกเป็นลิสต์ของสตริงย่อย โดยใช้ separator เป็นตัวคั่น (ค่าเริ่มต้นคือช่องว่าง)

ตัวอย่าง

```
"a,b,c".split(",") # ผลลัพธ์: ['a', 'b', 'c']
```

- replace(old, new)
 - แทนที่ old substring ด้วย new substring

ตัวอย่าง

```
"Hello World".replace("World", "Python") # ผลลัพธ์: 'Hello Python'
```

- find(substring)
 - คืนค่าดัชนีเริ่มต้นของ substring ที่พบครั้งแรก ถ้าไม่พบจะคืนค่า -1

ตัวอย่าง

```
"Hello".find("e") # ผลลัพธ์: 1
```

- startswith(prefix)
 - ตรวจสอบว่าสตริงขึ้นต้นด้วย prefix ที่กำหนดหรือไม่ (คืนค่า True/False)

ตัวอย่าง

```
"Hello".startswith("He") # ผลลัพธ์: True
```

- endswith(suffix)
 - ตรวจสอบว่าสตริงลงท้ายด้วย suffix ที่กำหนดหรือไม่ (คืนค่า True/False)

ตัวอย่าง

```
"Hello".endswith("lo") # ผลลัพธ์: True
```

ตัวอย่างการใช้งานเมธอดสตริง

```
text = " Hello, Python! "
print(text.strip())           # 'Hello, Python!'
print(text.lower())           # ' hello, python! '
print(text.upper())           # ' HELLO, PYTHON! '
print(text.replace("Python", "World")) # ' Hello, World! '
print(text.split(","))        # [' Hello', ' Python! ']
print(text.startswith(" He")) # True
print(text.find("Python"))     # 8
```

หมายเหตุ สามารถใช้ฟังก์ชัน `dir(obj)` เพื่อดูรายชื่อเมธอดและ attribute ทั้งหมดของอ็อบเจกต์ และใช้ `help(obj)` เพื่อดูคำอธิบายเมธอดและวิธีใช้งาน

ตัวอย่าง

```
s = "Hello"
print(dir(s))
```

ตัวอย่าง

```
help(str)
```

Formatted String Literals (F-string)

Formatted String Literals หรือ F-string คือคุณสมบัติที่ถูกนำมาใช้ใน Python 3.6 เพื่ออำนวยความสะดวกในการสร้างสตริงที่มีการจัดรูปแบบ โดยช่วยให้สามารถแทรกนิพจน์ (expressions) และตัวแปรลงในสตริงได้อย่างกระชับ อ่านง่าย และมีประสิทธิภาพสูงกว่าวิธีการจัดรูปแบบสตริงแบบดั้งเดิม

ลักษณะเด่นของ f-string

- สามารถฝังค่า ตัวแปร หรือ นิพจน์ ลงในสายอักขระได้โดยตรง
- อ่านและเขียนโค้ดได้ง่ายขึ้น
- รองรับการจัดรูปแบบตัวเลข วันที่ และค่าต่าง ๆ ได้หลากหลาย

Syntax ของ f-string

- เริ่มต้นสายอักขระด้วยตัวอักษร f หรือ F หน้าคำพูด ('...' หรือ "...")
- แทรกตัวแปรหรือนิพจน์ที่ต้องการใน {} ภายในสายอักขระนั้น

รูปแบบทั่วไป

f"ข้อความ {ตัวแปรหรือนิพจน์}"

ตัวอย่างการใช้งาน f-string

1. แทรกตัวแปรลงในสตริง

```
name = "Alice"
age = 25
print(f"ชื่อ: {name}, อายุ: {age} ปี")
# ผลลัพธ์: ชื่อ: Alice, อายุ: 25 ปี
```

2. แทรกนิพจน์หรือการคำนวณ

```
price = 59
quantity = 3
print(f"ราคารวม: {price * quantity} บาท")
# ผลลัพธ์: ราคารวม: 177 บาท
```

3.4 ชนิดข้อมูลรายการ (List)

List คือโครงสร้างข้อมูลประเภทหนึ่งที่อนุญาตให้เราสามารถจัดเก็บข้อมูลหลายรายการไว้ในตัวแปรเดียวได้ ตัวอย่างเช่น หากต้องการจัดเก็บอายุของนักเรียนทุกคนในชั้นเรียน สามารถใช้ List เพื่อทำงานนี้ได้อย่างมีประสิทธิภาพ

List ในภาษา Python มีลักษณะคล้ายกับ อาร์เรย์ (Array) ในภาษาโปรแกรมอื่น ๆ แต่มีความยืดหยุ่นมากกว่า กล่าวคือ List สามารถจัดเก็บข้อมูลได้หลายประเภท (Data Types) ภายใน List เดียวกัน ไม่ว่าจะเป็นตัวเลข ตัวอักษร หรือข้อมูลประเภทอื่น ๆ

คุณสมบัติหลักของ Python List

1. มีลำดับ (Ordered) ข้อมูลที่ถูกเพิ่มเข้าไปใน List จะถูกจัดเก็บตามลำดับ และสามารถเข้าถึงข้อมูลได้ผ่านดัชนี (index)
2. เปลี่ยนแปลงได้ (Mutable) สามารถเพิ่ม ลบ หรือแก้ไขข้อมูลภายใน List ได้หลังจากการสร้าง
3. ยอมรับค่าซ้ำ (Allows Duplicates) สามารถมีข้อมูลที่มีค่าซ้ำกันใน List เดียวกันได้
4. เก็บข้อมูลได้หลายชนิด (Heterogeneous) สามารถเก็บข้อมูลที่มีชนิดแตกต่างกันได้ เช่น ตัวเลข ข้อความ (string) หรือแม้กระทั่ง List อื่น ๆ
5. มีฟังก์ชันและเมธอด (Method) สำหรับจัดการข้อมูลใน List อย่างหลากหลาย

ตัวอย่างการใช้งาน List

```
# การสร้าง List เพื่อเก็บอายุนักเรียนในชั้นเรียน
student_ages = [15, 16, 15, 17, 16]
print(student_ages)
```

การสร้าง List

การสร้าง List ในภาษา Python เป็นเรื่องง่ายและทำได้หลายวิธี

1. สร้าง List ว่าง (Empty List) เราสามารถสร้าง List ที่ไม่มีสมาชิกได้ โดยใช้เพียงวงเล็บเหลี่ยม [] (square brackets)

ตัวอย่าง

```
my_list = []
print(type(my_list))
```

2. สร้าง List พร้อมสมาชิก เราสามารถกำหนดสมาชิกเริ่มต้นให้กับ List ได้ทันที โดยใช้สมาชิกเหล่านั้นไว้ในวงเล็บเหลี่ยม [] (square brackets) และคั่นด้วยเครื่องหมายจุลภาค , (commas)

ตัวอย่าง

```
# List ที่มีสมาชิกเป็นตัวเลข
numbers = [1, 2, 3, 4, 5]

# List ที่มีสมาชิกเป็นข้อความ (strings)
fruits = ["apple", "banana", "cherry"]

# List ที่มีสมาชิกหลากหลายชนิด (heterogeneous)
mixed_list = [10, "hello", 3.14, True]
```

3. การสร้าง List โดยใช้คอนสตรักเตอร์ list()

ตัวอย่าง

```
empty_list = list()
print(empty_list)
```

การสร้างลิสต์จากข้อมูลประเภทลำดับอื่น (Iterable)

ตัวอย่าง

```
# แปลงจาก tuple
my_tuple = (1, 2, 3)
list_from_tuple = list(my_tuple)
print(list_from_tuple)

# แปลงจาก string
my_string = "python"
list_from_string = list(my_string)
print(list_from_string)

# แปลงจาก range
list_from_range = list(range(5))
print(list_from_range)
```

การเข้าถึงสมาชิกใน List (Accessing List Elements)

ดัชนีบวก (Positive Indexing)

สมาชิกแต่ละตัวใน List มีตำแหน่งที่เรียกว่า ดัชนี (index) ซึ่งเริ่มต้นจาก 0 เราสามารถเข้าถึงหรือดึงค่าของสมาชิกได้โดยใช้ชื่อ List ตามด้วยดัชนีในวงเล็บเหลี่ยม



รูป 2 ตัวอย่างดัชนีของสมาชิกในลิสต์ (Index of List Element)



รูป 3 ตัวอย่างการเข้าถึงสมาชิกในลิสต์ด้วยดัชนีบวก (Positive Indexing)

ตัวอย่าง

```
languages = ['Python', 'Swift', 'C++']

# access the first element
print('languages[0] =', languages[0])

# access the third element
print('languages[2] =', languages[2])
```

ดัชนีลบ (Negative Indexing)

Negative Indexing คือความสามารถในการเข้าถึงสมาชิกใน List โดยนับจากท้าย List ย้อนกลับไปข้างหน้า

โดยปกติแล้ว Python List จะใช้ดัชนีเริ่มต้นจาก 0 (สำหรับสมาชิกตัวแรก) ไปจนถึง N-1 (สำหรับสมาชิกตัวสุดท้าย) แต่ Negative Indexing จะทำงานตรงกันข้าม โดย

- ดัชนี -1 หมายถึงสมาชิกตัวสุดท้ายของ List
- ดัชนี -2 หมายถึงสมาชิกตัวรองสุดท้ายของ List
- ดัชนี -3 หมายถึงสมาชิกตัวที่สามจากท้าย List
- และเป็นเช่นนี้ไปเรื่อย ๆ

	['Python', 'Swift', 'C++']		
Index →	0	1	2
Negative Index →	-3	-2	-1

รูป 4 ตัวอย่างการเข้าถึงสมาชิกในลิสต์ด้วยดัชนีลบ (Negative Indexing)

ตัวอย่าง

```
languages = ['Python', 'Swift', 'C++']

# access the last item
print('languages[-1] =', languages[-1])

# access the third last item
print('languages[-3] =', languages[-3])
```

ตัวอย่าง

```
fruits = ["apple", "banana", "cherry", "orange", "kiwi"]

last_fruit = fruits[-1]
print(f"The last fruit is: {last_fruit}")

second_last_fruit = fruits[-2]
print(f"The second last fruit is: {second_last_fruit}")
```

การใช้ Negative Indexing มีประโยชน์มากเมื่อเราต้องการเข้าถึงสมาชิกตัวท้าย ๆ ของ List โดยไม่ต้องทราบความยาวทั้งหมดของ List

Slicing List

Slicing คือการแบ่งส่วนของ List เพื่อเข้าถึงสมาชิกส่วนใดส่วนหนึ่งของ List โดยใช้เครื่องหมาย : (colon) เป็นตัวดำเนินการหลัก

รูปแบบการ Slicing

`list_name[start: stop: step]`

- start: ตำแหน่งเริ่มต้น (index) ที่ต้องการ Slicing (ถ้าไม่ระบุจะเริ่มที่ 0)
- stop :ตำแหน่งสุดท้ายของการ Slicing (ไม่รวมสมาชิกที่ดัชนีนี้)
- step: ระยะห่างระหว่างแต่ละค่าที่ถูกเลือก (ถ้าไม่ระบุจะเป็น 1)

การใช้งาน Slicing

Python Slicing มีความยืดหยุ่นสูง โดยสามารถละเว้น start, stop, หรือ step ได้ ซึ่งการละเว้นจะทำให้ Python ใช้ค่าเริ่มต้น

1. การดึงสมาชิกทั้งหมด (Get all the Items)

หากละเว้นทั้ง start และ stop จะเป็นการคัดลอก List ทั้งหมด

ตัวอย่าง

```
my_list = [1, 2, 3, 4, 5]
# ดึงสมาชิกทั้งหมด
all_items = my_list[:]
print(all_items)
```

2. ดึงสมาชิกจากตำแหน่งเริ่มต้นถึงท้าย (Get all the Items After a Specific Position)

หากละเว้น stop การ Slicing จะเริ่มต้นจาก start ที่กำหนด และดำเนินต่อไปจนถึงสมาชิกตัวสุดท้ายของ List

ตัวอย่าง

```
my_list = ["a", "b", "c", "d", "e"]
# ดึงสมาชิกตั้งแต่ดัชนี 2 (คือ "c") จนถึงท้าย List
items_from_2 = my_list[2:]
print(items_from_2)
```

3. ดึงสมาชิกตั้งแต่ต้นจนถึงตำแหน่งสิ้นสุด (Get all the Items Before a Specific Position)

หากละเว้น start การ Slicing จะเริ่มต้นจากสมาชิกตัวแรก (ดัชนี 0) และสิ้นสุดที่ดัชนี stop ที่กำหนด (ไม่รวมดัชนี stop)

ตัวอย่าง

```
my_list = ["a", "b", "c", "d", "e"]
# ดึงสมาชิกตั้งแต่ต้น List จนถึงดัชนี 3 (ไม่รวม "d")
items_to_3 = my_list[:3]
print(items_to_3)
```

4. ดึงสมาชิกจากตำแหน่งหนึ่งถึงอีกตำแหน่งหนึ่ง (Get all the Items from One Position to Another Position)

การกำหนดทั้ง start และ stop เพื่อระบุช่วงที่ต้องการดึงข้อมูล

ตัวอย่าง

```
my_list = ["a", "b", "c", "d", "e"]
# ดึงสมาชิกตั้งแต่ดัชนี 1 (คือ "b") จนถึงดัชนี 4 (ไม่รวม "e")
items_1_to_4 = my_list[1:4]
print(items_1_to_4)
```

5. ดึงสมาชิกตาม step (Get the Items at Specified Intervals)

การใช้ step เพื่อกำหนดช่วงในการเลือกสมาชิก โดยค่าเริ่มต้นของ step คือ 1

ตัวอย่าง

```
my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
# ดึงสมาชิกตั้งแต่ต้นจนจบ โดยก้าวทีละ 2
items_step_2 = my_list[::2]
# ผลลัพธ์: [0, 2, 4, 6, 8]
print(items_step_2)
```

การใช้ Negative Indexing ใน Slicing

เราสามารถใช้ดัชนีติดลบ (Negative Indexing) ในการ Slicing ได้เช่นกัน ซึ่งมีประโยชน์ในการนับจากท้าย List

ตัวอย่าง

```
my_list = [10, 20, 30, 40, 50]
last_two = my_list[-2:] # ดึงสมาชิก 2 ตัวสุดท้าย
print(last_two) # ผลลัพธ์: [40, 50]
```

การเพิ่มสมาชิกใน List

Python List มีคุณสมบัติที่สามารถเปลี่ยนแปลงได้ (Mutable) ทำให้เราสามารถเพิ่ม ลบ หรือแก้ไขสมาชิกภายใน List ได้ตลอดเวลา การเพิ่มสมาชิกสามารถทำได้หลายวิธี ขึ้นอยู่กับตำแหน่งที่ต้องการเพิ่ม

1. การเพิ่มสมาชิกที่ท้าย List (Appending)

สามารถเพิ่มสมาชิกใหม่เข้าไปที่ท้าย List ได้ด้วยเมธอด `append()`

ตัวอย่าง

```
# สร้าง List เริ่มต้น
fruits = ["apple", "banana", "cherry"]

# เพิ่ม "orange" เข้าไปที่ท้าย List
fruits.append("orange")

print('Updated List:', fruits)
# ผลลัพธ์: ['apple', 'banana', 'cherry', 'orange']
```

2. การเพิ่มสมาชิกที่ดัชนีที่ระบุ (Inserting)

หากต้องการแทรกสมาชิกที่ตำแหน่งดัชนีที่ต้องการ เราจะใช้เมธอด `insert()`

เมธอด `insert()` รับสองพารามิเตอร์ `insert(index, element)`

- `index`: ตำแหน่งดัชนีที่ต้องการแทรกสมาชิก
- `element`: สมาชิกที่ต้องการเพิ่ม

ตัวอย่าง

```
# สร้าง List ของตัวเลข
numbers = [1, 2, 4, 5]

# แทรก 3 ที่ดัชนี 2
numbers.insert(2, 3)

print(numbers)
```

ตัวอย่าง

```
fruits = ['apple', 'banana', 'orange']
print("Original List:", fruits)

fruits.insert(2, 'cherry')

print("Updated List:", fruits)
```

3. การเพิ่มสมาชิกจาก Iterables อื่น ๆ (Extending)

เมธอด `extend()` ใช้สำหรับการเพิ่มสมาชิกทั้งหมดจาก iterable อื่น ๆ เช่น List, Tuple, Dictionary หรือ String เข้าไปต่อท้าย List ปัจจุบัน

ต่างจาก `append()` ตรงที่ `extend()` จะเพิ่มสมาชิกทีละตัวจาก iterable นั้น ๆ เข้าไปใน List เดิม

Iterable คือวัตถุ (object) ในภาษา Python ที่สามารถ วนซ้ำ (iterated over) ได้ หมายความว่าเราสามารถเข้าถึงสมาชิกภายในวัตถุนั้นทีละตัวได้

ในทางปฏิบัติ iterable คือโครงสร้างข้อมูลที่สามารถใช้ในลูป `for` ได้

ตัวอย่าง

```
# List หลัก
list1 = [1, 2, 3]

# เพิ่มสมาชิกจาก List ที่สอง (iterable)
list2 = [4, 5, 6]
list1.extend(list2)

print(list1)
# ผลลัพธ์: [1, 2, 3, 4, 5, 6]
```

ตัวอย่างความแตกต่างระหว่าง `append()` กับ `extend()`

ตัวอย่างการใช้ `extend()`

```
list_a = [1, 2, 3]
list_b = [4, 5]
```

```
# ใช้ extend() เพื่อเพิ่มสมาชิกจาก list_b เข้าไปใน list_a
list_a.extend(list_b)

print(list_a)
# ผลลัพธ์: [1, 2, 3, 4, 5] สมาชิก 4 และ 5 ถูกเพิ่มเข้ามาทีละตัวใน list_a
```

ตัวอย่างการใช้ append()

```
list_a = [1, 2, 3]
list_b = [4, 5]

# ใช้ append() เพื่อเพิ่ม list_b เข้าไปใน list_a
list_a.append(list_b)

print(list_a)
# ผลลัพธ์: [1, 2, 3, [4, 5]] list_b ถูกเพิ่มเป็นสมาชิกตัวเดียวใน list_a
```

การแก้ไขสมาชิกใน List

เนื่องจาก Python List เป็นโครงสร้างข้อมูลที่สามารถเปลี่ยนแปลงได้ (mutable) เราจึงสามารถแก้ไขค่าของสมาชิกใน List ได้โดยตรง โดยใช้วงเล็บเหลี่ยม (square brackets) [] เพื่อระบุตำแหน่งดัชนี (index) และใช้เครื่องหมาย = ในการกำหนดค่าใหม่

หลักการแก้ไขสมาชิก

การแก้ไขสมาชิกทำได้โดยระบุชื่อ List ตามด้วยดัชนีของสมาชิกที่ต้องการแก้ไข จากนั้นกำหนดค่าใหม่

```
list_name[index] = new_value
```

ตัวอย่าง

```
colors = ['Red', 'Black', 'Green']
print('Original List:', colors)
# ผลลัพธ์: Original List: ['Red', 'Black', 'Green']

# แก้ไขสมาชิกตัวแรก (ดัชนี 0) จาก 'Red' เป็น 'Purple'
colors[0] = 'Purple'
```

```
# แก้ไขสมาชิกตัวที่สาม (ดัชนี 2) จาก 'Green' เป็น 'Blue'
colors[2] = 'Blue'

print('Updated List:', colors)
# ผลลัพธ์: Updated List: ['Purple', 'Black', 'Blue']
```

การแก้ไขหลายสมาชิกพร้อมกัน (Slicing and Assignment)

นอกจากแก้ไขสมาชิกทีละตัวแล้ว ยังสามารถแก้ไขสมาชิกหลายตัวพร้อมกันได้โดยใช้เทคนิค Slicing ร่วมกับการกำหนดค่าใหม่

เราสามารถกำหนดค่าใหม่ให้กับช่วงของ List ที่เลือกไว้ (slice) โดย List ใหม่ที่กำหนดเข้ามาไม่จำเป็นต้องมีความยาวเท่ากับช่วงเดิม

ตัวอย่าง

```
# List เดิม
numbers = [10, 20, 30, 40, 50]

# แก้ไขสมาชิกตั้งแต่ดัชนี 1 ถึง 3 (ไม่รวม 3) คือ 20 และ 30
# ให้เป็น [99, 100]
numbers[1:3] = [99, 100]

print(numbers)
# ผลลัพธ์: [10, 99, 100, 40, 50]
```

การลบสมาชิกออกจาก List

สามารถลบสมาชิกได้หลายวิธี ขึ้นอยู่กับว่าต้องการลบสมาชิกโดยระบุค่า (value) หรือโดยระบุตำแหน่งดัชนี (index)

1. การลบสมาชิกโดยระบุค่าด้วย remove()

เมธอด remove() ใช้สำหรับลบสมาชิกตัวแรกที่มีค่าตรงกับที่ระบุ

- หากมีสมาชิกที่มีค่าซ้ำกันอยู่ใน List, remove() จะลบเฉพาะตัวแรกที่พบ
- หากไม่พบค่าที่ระบุใน List จะเกิดข้อผิดพลาด ValueError

ตัวอย่าง

```
fruits = ["apple", "banana", "cherry", "banana"]

# ลบสมาชิกที่มีค่า 'banana' ตัวแรกที่พบ
fruits.remove("banana")

print(fruits) # ผลลัพธ์: ['apple', 'cherry', 'banana']
```

2. การลบสมาชิกโดยระบุตำแหน่งด้วย del Statement

นอกเหนือจาก remove() เราสามารถใช้คำสั่ง del (delete) เพื่อลบสมาชิกในตำแหน่งที่ต้องการ

del เป็นคำสั่งที่ยืดหยุ่น สามารถใช้ลบสมาชิกหนึ่งตัว สมาชิกหลายตัว หรือแม้กระทั่งลบทั้ง List ได้

การลบสมาชิกหนึ่งตัว

ใช้ del ตามด้วยชื่อ List และดัชนีของสมาชิกที่ต้องการลบ

ตัวอย่าง

```
numbers = [1, 2, 3, 4, 5]

del numbers[2] # ลบสมาชิกที่ดัชนี 2 (คือ 3)

print(numbers) # ผลลัพธ์: [1, 2, 4, 5]
```

การลบสมาชิกหลายตัว (Slicing)

ใช้ del ร่วมกับ Slicing เพื่อลบสมาชิกในช่วงที่กำหนด

ตัวอย่าง

```
numbers = [1, 2, 3, 4, 5, 6]

del numbers[1:4] # ลบสมาชิกตั้งแต่ดัชนี 1 ถึง 3 (คือ 2, 3, 4)

print(numbers) # ผลลัพธ์: [1, 5, 6]
```

การลบ List

ใช้ `del` เพื่อลบ List ทั้งหมดออกจากหน่วยความจำ

ตัวอย่าง

```
my_list = [1, 2, 3]

del my_list # ลบ List 'my_list'

print(my_list) # หากพยายามเรียกใช้ my_list จะเกิด NameError
```

การลบสมาชิกด้วย pop()

เมธอด `pop()` ใช้ในการลบสมาชิกตามดัชนีที่ระบุ และส่งคืนค่าของสมาชิกนั้น หากไม่ระบุพารามิเตอร์ใด ๆ `pop()` จะลบสมาชิกตัวสุดท้ายของ List

ตัวอย่าง

```
stack = ["A", "B", "C"]

# ลบสมาชิกตัวสุดท้าย (C) และเก็บค่าไว้ในตัวแปร
removed_item = stack.pop()

print(removed_item) # ผลลัพธ์: C
print(stack)        # ผลลัพธ์: ['A', 'B']
```

การหาความยาวของ List ด้วย len()

ในภาษา Python เราสามารถใช้ฟังก์ชัน `len()` (built-in function) เพื่อหาจำนวนสมาชิกทั้งหมด หรือความยาวของ List

ตัวอย่าง

```
# List ของตัวเลข
numbers = [10, 20, 30, 40, 50]

# ใช้ len() เพื่อหาความยาว
list_length = len(numbers)

print(f"The length of the list is: {list_length}") # ผลลัพธ์: The length of the list is: 5
```


การตรวจสอบการเป็นสมาชิกใน List

in และ not in ใช้เพื่อตรวจสอบว่าสมาชิกที่ต้องการค้นหาอยู่ใน List หรือไม่

- in: คืนค่า True ถ้าสมาชิกมีอยู่ใน List
- not in: คืนค่า True ถ้าสมาชิกไม่มีอยู่ใน List

ตัวอย่าง

```
# สร้าง List
students = ['Alice', 'Bob', 'Charlie', 'David']

# ใช้ 'in' เพื่อตรวจสอบว่า 'Alice' มีอยู่ใน List หรือไม่
print('Alice' in students)
# ผลลัพธ์: True

# ใช้ 'in' เพื่อตรวจสอบว่า 'Eve' มีอยู่ใน List หรือไม่
print('Eve' in students)
# ผลลัพธ์: False

# ใช้ 'not in' เพื่อตรวจสอบว่า 'Eve' ไม่อยู่ใน List หรือไม่
print('Eve' not in students)
# ผลลัพธ์: True
```

การวนซ้ำ (Iteration) ใน List ด้วย for Loop

การวนซ้ำ หรือ iteration คือกระบวนการเข้าถึงสมาชิกแต่ละตัวใน List ทีละตัว ตั้งแต่ต้นจนจบ โดยทั่วไป เราใช้ for loop เพื่อทำงานนี้

ตัวอย่าง

```
fruits = ['apple', 'banana', 'orange']

# iterate through the list
for fruit in fruits:
    print(fruit)
```

การวนซ้ำด้วย for loop และดัชนี (Index)

หากต้องการเข้าถึงทั้งสมาชิกและดัชนีของสมาชิกในขณะที่วนซ้ำ สามารถใช้ฟังก์ชัน `range()` ร่วมกับ `len()` หรือใช้ฟังก์ชัน `enumerate()`

การใช้ `range(len(list_name))`

`range()` เป็นฟังก์ชันใน Python ที่ใช้สำหรับ สร้างลำดับของตัวเลข โดยปกติแล้วจะใช้ร่วมกับ `for loop` เพื่อวนซ้ำ (iterate) ตามจำนวนครั้งที่กำหนด หรือวนซ้ำในช่วงของตัวเลขที่ต้องการ

ตัวอย่างการใช้ `range(len(list_name))`

```
colors = ['Red', 'Green', 'Blue']
# วนซ้ำตามดัชนี (0, 1, 2)
for i in range(len(colors)):
    print(f"Index {i}: {colors[i]}")
```

การใช้ `enumerate()`

ฟังก์ชัน `enumerate()` จะส่งคืนทั้งดัชนีและค่าของสมาชิก ทำให้โค้ดกระชับและอ่านง่ายขึ้น `enumerate()` เป็นฟังก์ชันใน Python ที่ใช้สำหรับ วนซ้ำ (iterate) ไปพร้อมกับดึงทั้งค่า (value) และดัชนี (index) ของแต่ละองค์ประกอบในโครงสร้างข้อมูลที่สามารถวนซ้ำได้ (iterable) เช่น List, Tuple, String หรือ Dictionary

เมื่อใช้ `for loop` โดยตรงกับ List จะเข้าถึงได้แค่ค่าขององค์ประกอบนั้น ๆ

รูปแบบการใช้งานของ `enumerate()`

`enumerate(iterable, start=0)`

- `iterable`: โครงสร้างข้อมูลที่คุณต้องการวนซ้ำ (เช่น List, Tuple)
- `start`: (Optional) ดัชนีเริ่มต้นที่ต้องการ ค่าเริ่มต้นคือ 0

ตัวอย่างการใช้ `enumerate()`

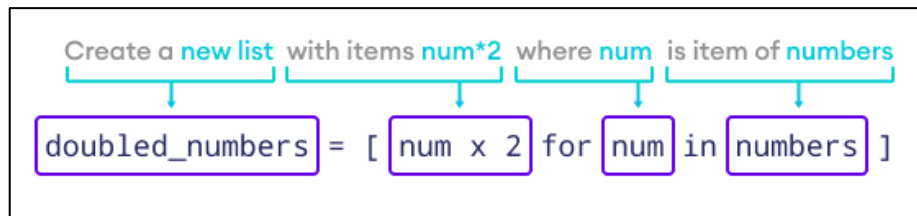
```
colors = ['Red', 'Green', 'Blue']
# วนซ้ำพร้อมรับดัชนีและค่า
for index, color in enumerate(colors, start=1):
    print(f"Index {index}: {color}")
```

List Comprehension

List Comprehension คือ วิธีการสร้าง List ใหม่ขึ้นมาอย่างย่อ โดยอาศัยข้อมูลจาก List เดิมที่มีอยู่แล้ว เป็นวิธีที่ช่วยให้โค้ดสั้นกระชับและอ่านง่ายขึ้น

ตัวอย่างการใช้งาน

สมมติว่ามี List ที่เก็บตัวเลขและต้องการสร้าง List ใหม่ที่แต่ละสมาชิกมีค่าเป็นสองเท่าของสมาชิกใน List เดิม



รูป 5 ตัวอย่างการสร้าง List Comprehension

Syntax of List Comprehension

[expression for item in list]

- **expression** คือ นิพจน์หรือการดำเนินการกับ item เพื่อสร้างเป็นสมาชิกใน List ใหม่ (เช่น `num * 2`)
- **for item in list** คือ การวนลูป (iteration) เพื่อเข้าถึงสมาชิกแต่ละตัว (item) ใน List ตั้งต้น (list)

ตัวอย่าง

```
# List ตั้งต้น
numbers = [1, 2, 3, 4]

# การใช้ List Comprehension เพื่อสร้าง List ใหม่
doubled_numbers = [num * 2 for num in numbers]

# แสดงผลลัพธ์
print(doubled_numbers)
```

Conditionals in List Comprehension

สามารถใช้คำสั่งเงื่อนไข เช่น `if` และ `if...else` เพื่อเพิ่มความสามารถให้กับการสร้าง List

ใช้ if สำหรับคัดกรองข้อมูล (Filtering)

รูปแบบนี้เป็นการใช้ if เพื่อกำหนดเงื่อนไขว่า สมาชิกตัวใดบ้างจาก List เดิมที่จะถูกนำไปประมวลผลและเพิ่มเข้าไปใน List ใหม่ กล่าวคือ สมาชิกที่ไม่ตรงตามเงื่อนไขจะถูกคัดทิ้งไป

Syntax

[expression for item in list if condition]

- **if condition** เป็นเงื่อนไขสำหรับตรวจสอบ item แต่ละตัว โดย expression จะถูกประมวลผลและเพิ่มเข้าไปใน List ใหม่ ก็ต่อเมื่อ item นั้นผ่านเงื่อนไข (condition เป็น True) เท่านั้น

ตัวอย่างการใช้ if สร้าง List ใหม่เฉพาะเลขคู่

```
# filtering even numbers from a list
even_numbers = [num for num in range(1, 10) if num % 2 == 0 ]

print(even_numbers)
```

ตัวอย่างการใช้ if สร้าง List ใหม่เฉพาะเลขคู่ที่ถูกคูณด้วย 2

```
# List ดั้งเดิม
numbers = [1, 2, 3, 4, 5, 6]

# สร้าง List ใหม่โดยเลือกเฉพาะเลขคู่ (num % 2 == 0) แล้วคูณด้วย 2
doubled_even_numbers = [num * 2 for num in numbers if num % 2 == 0]

print(doubled_even_numbers)
```

การใช้ if...else

ในรูปแบบนี้ จะใช้เงื่อนไข if...else เพื่อเลือกว่าจะ แปลงค่าสมาชิกแต่ละตัว ด้วยนิพจน์ใด ซึ่งหมายความว่า สมาชิกทุกตัวจาก List เดิมจะถูกนำมาประมวลผลทั้งหมด ไม่มีการคัดทิ้ง แต่ผลลัพธ์ใน List ใหม่จะแตกต่างกันไปตามเงื่อนไข

Syntax

[expression_if_true if condition else expression_if_false for item in iterable]

ข้อแตกต่าง คือ นิพจน์ if...else ทั้งหมดจะถูกย้ายมาวางไว้ที่ส่วนหน้าสุด ก่อน for loop

ตัวอย่าง

```
numbers = [1, 2, 3, 4, 5, 6]

# find even and odd numbers
even_odd_list = ["Even" if i % 2 == 0 else "Odd" for i in numbers]

print(even_odd_list)
```

for Loop vs. List Comprehension

ตัวอย่างความแตกต่างการใช้ for Loop vs. List Comprehension

ตัวอย่าง for Loop

```
numbers = [1, 2, 3, 4, 5]
square_numbers = []

# for loop to square each elements
for num in numbers:
    square_numbers.append(num * num)

print(square_numbers)
# Output: [1, 4, 9, 16, 25]
```

ตัวอย่าง List Comprehension

```
numbers = [1, 2, 3, 4, 5]

# create a new list using list comprehension
square_numbers = [num * num for num in numbers]

print(square_numbers)
# Output: [1, 4, 9, 16, 25]
```

3.5 ชนิดข้อมูลทูเปิล (Tuple)

Tuple คือโครงสร้างข้อมูลประเภทหนึ่งในภาษา Python ที่ใช้เก็บข้อมูลหลายรายการไว้ในตัวแปรเดียว Tuple มีลักษณะคล้ายกับ List แต่มีความแตกต่างที่สำคัญคือ Tuple ไม่สามารถแก้ไขได้ (immutable) เมื่อสร้างขึ้นแล้ว

คุณสมบัติหลักของ Tuple

1. มีลำดับ (Ordered) สมาชิกใน Tuple จะถูกจัดเก็บตามลำดับที่เรากำหนด และสามารถเข้าถึงได้ผ่านดัชนี (index)
2. ไม่สามารถเปลี่ยนแปลงได้ (Immutable) นี่คือการแตกต่างที่สำคัญที่สุด เมื่อ Tuple ถูกสร้างขึ้น เราไม่สามารถเพิ่ม ลบ หรือแก้ไขสมาชิกภายในได้
3. ยอมรับค่าซ้ำ (Allows Duplicates) สามารถมีสมาชิกที่มีค่าซ้ำกันได้
4. เก็บข้อมูลได้หลากหลายชนิด (Heterogeneous) สามารถเก็บข้อมูลต่างชนิดกันได้ เช่น ตัวเลข ข้อความ หรือ List

การสร้าง Tuple

การสร้าง Tuple โดยใช้วงเล็บ ()

สามารถสร้าง Tuple ได้โดยการนำสมาชิกมาใส่ไว้ในวงเล็บ () และคั่นด้วยเครื่องหมายจุลภาค , (comma)

ตัวอย่างการสร้าง Tuple

```
my_tuple = ("apple", "banana", "cherry", 123)
single_tuple = ("hello",) # การสร้าง Tuple ที่มีสมาชิกเพียงตัวเดียว ต้องมีเครื่องหมายจุลภาคต่อท้าย
print(type(single_tuple))
```

การสร้าง Tuple โดยใช้ tuple() constructor

tuple() constructor เป็นฟังก์ชันที่ใช้สร้าง Tuple โดยรับ iterable (เช่น List, String, หรือ Tuple อื่น ๆ) เป็นอาร์กิวเมนต์

ตัวอย่างการใช้ tuple() constructor เพื่อสร้าง Tuple จาก List หรือ iterable อื่น ๆ

```
tuple_constructor = tuple(['Jack', 'Maria', 'David'])
print(tuple_constructor) # ผลลัพธ์: ('Jack', 'Maria', 'David')
```

Tuple ว่าง (Empty Tuple)

สามารถสร้าง Tuple ที่ไม่มีสมาชิกได้โดยใช้เพียงวงเล็บเปล่า () หรือใช้ tuple() constructor ที่ไม่มีอาร์กิวเมนต์

ตัวอย่างการสร้าง Tuple ว่าง

```
empty_tuple = ()
print(empty_tuple)
```

ตัวอย่างการสร้าง Tuple ว่าง

```
empty_tuple = tuple()
print(empty_tuple)
```

Tuple ที่มีสมาชิกหลากหลายชนิด (Mixed Data Types)

Tuple สามารถเก็บสมาชิกที่มีชนิดข้อมูลต่างกันได้ ใน Tuple เดียวกัน เช่น การรวม String, Integer, และ Boolean ไว้ด้วยกัน

ตัวอย่าง Tuple ที่มีข้อมูลหลากหลายชนิด

```
mixed_tuple = (2, 'Hello', 'Python', True)
print(mixed_tuple)
# ผลลัพธ์: (2, 'Hello', 'Python', True)
```

ข้อดีของ Tuple

เนื่องจาก Tuple ไม่สามารถเปลี่ยนแปลงได้ ทำให้มีข้อดีในด้านประสิทธิภาพและความปลอดภัยของข้อมูล

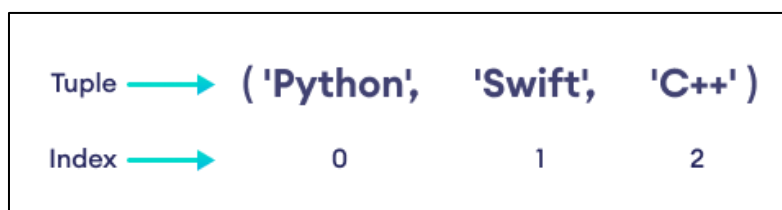
1. ประสิทธิภาพ Tuple มักจะทำงานเร็วกว่า List เล็กน้อย
2. ความปลอดภัยของข้อมูล เหมาะสำหรับเก็บข้อมูลที่เราต้องการให้คงที่ เช่น ค่าคงที่ (constants) หรือข้อมูลการตั้งค่า (configuration data)
3. การใช้งานใน Dictionary Tuple สามารถใช้เป็น Key ใน Dictionary ได้ (เนื่องจากเป็น immutable) ในขณะที่ List ไม่สามารถทำได้

การเข้าถึงสมาชิกใน Python Tuple

การเข้าถึงสมาชิกใน Tuple ทำได้โดยใช้ ดัชนี (index) ซึ่งเป็นหมายเลขตำแหน่งของสมาชิกแต่ละตัวใน Tuple

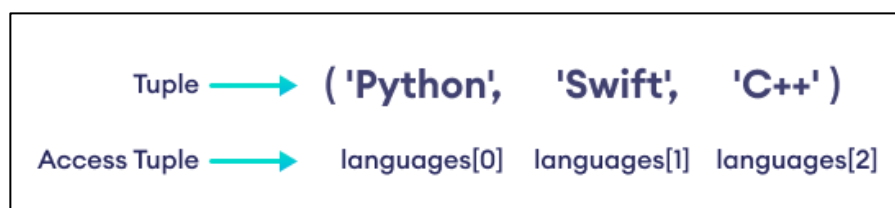
หลักการของดัชนีใน Tuple

- ดัชนีของ Tuple จะเริ่มต้นจาก 0 เสมอ
- สมาชิกตัวแรกมีดัชนีเป็น 0
- สมาชิกตัวที่สองมีดัชนีเป็น 1
- และเป็นเช่นนี้ต่อไปเรื่อย ๆ



รูป 6 ดัชนีใน Tuple

การเข้าถึงสมาชิกด้วยดัชนี เราใช้เครื่องหมายวงเล็บเหลี่ยม [] ตามด้วยหมายเลขดัชนี เพื่อเข้าถึงสมาชิกที่ต้องการ



รูป 7 การเข้าถึงสมาชิกด้วยดัชนีใน Tuple

ตัวอย่าง

```
languages = ('Python', 'Swift', 'C++')

# เข้าถึงสมาชิกตัวแรก (ดัชนี 0)
print(languages[0]) # ผลลัพธ์: Python

# เข้าถึงสมาชิกตัวที่สาม (ดัชนี 2)
print(languages[2]) # ผลลัพธ์: C++
```


Negative Indexing

นอกจากการใช้ดัชนีบวก (นับจากต้น) ยังสามารถใช้ Negative Indexing (ดัชนีติดลบ) เพื่อเข้าถึงสมาชิกโดยนับจากท้าย Tuple ได้

- ดัชนี -1 หมายถึงสมาชิกตัวสุดท้าย
- ดัชนี -2 หมายถึงสมาชิกตัวรองสุดท้าย
- และเป็นเช่นนี้ต่อไป

ตัวอย่าง

```
languages = ('Python', 'Swift', 'C++')

# เข้าถึงสมาชิกตัวสุดท้าย (ดัชนี -1)
print(languages[-1]) # ผลลัพธ์: C++

# เข้าถึงสมาชิกตัวรองสุดท้าย (ดัชนี -2)
print(languages[-2]) # ผลลัพธ์: Swift
```

Tuple ไม่สามารถแก้ไขได้ (Immutable)

Tuple เป็นโครงสร้างข้อมูลที่ไม่สามารถเปลี่ยนแปลงได้ (immutable) นี่หมายความว่าเมื่อ Tuple ถูกสร้างขึ้น เราไม่สามารถเพิ่ม (add) เปลี่ยนแปลง (change) หรือลบ (delete) สมาชิกภายในได้

ลักษณะเฉพาะของการแก้ไขไม่ได้ หากพยายามแก้ไขสมาชิกใน Tuple โดยการกำหนดค่าใหม่ให้กับดัชนีใด ๆ คุณจะได้รับข้อผิดพลาด TypeError

ตัวอย่าง

```
cars = ('BMW', 'Tesla', 'Ford', 'Toyota')

# พยายามแก้ไขสมาชิกตัวแรก (ดัชนี 0) จะเกิดข้อผิดพลาด TypeError
cars[0] = 'Nissan'
```

ความยาวของ Tuple

สามารถใช้ฟังก์ชัน len() เพื่อหาจำนวนสมาชิกทั้งหมด (ความยาว) ของ Tuple ได้ ฟังก์ชันนี้ทำงานในลักษณะเดียวกับการหาความยาวของ List

ตัวอย่าง

```
cars = ('BMW', 'Tesla', 'Ford', 'Toyota')

# ใช้ len() เพื่อหาจำนวนสมาชิกใน Tuple
total_items = len(cars)

print(f"Total Items: {total_items}")
# ผลลัพธ์: Total Items: 4
```

การวนซ้ำ (Iterate) ใน Tuple

สามารถวนซ้ำเพื่อเข้าถึงสมาชิกแต่ละตัวใน Tuple ได้โดยใช้ for loop ซึ่งวิธีการนี้เหมือนกับ การวนซ้ำผ่าน List

การใช้งาน for loop กับ Tuple

for loop จะทำงานผ่านสมาชิกใน Tuple ทีละตัวตั้งแต่ต้นจนจบ ทำให้เราสามารถ ประมวลผลหรือแสดงผลข้อมูลแต่ละรายการได้

ตัวอย่าง

```
fruits = ('apple', 'banana', 'orange')

# วนซ้ำผ่านสมาชิกใน Tuple fruits
for fruit in fruits:
    print(fruit)
```

การวนซ้ำด้วยดัชนี (index)

หากต้องการเข้าถึงสมาชิกพร้อมกับดัชนี (index) ของสมาชิกใน Tuple เราสามารถใช้ ฟังก์ชัน range() ร่วมกับ len() หรือใช้ฟังก์ชัน enumerate()

ตัวอย่างการใช้ range() และ len()

```
languages = ('Python', 'Java', 'C#')
# วนซ้ำตามดัชนี (0, 1, 2)
for i in range(len(languages)):
    print(f"Index {i}: {languages[i]}")
```

ตัวอย่างการใช้ enumerate()

```
languages = ('Python', 'Java', 'C#')

# ใช้ enumerate() เพื่อรับทั้งดัชนีและค่าของสมาชิก
for index, language in enumerate(languages, 1):
    print(f"Index {index}: {language}")
```

การตรวจสอบว่าสมาชิกมีอยู่ใน Tuple หรือไม่

ใน Python เราสามารถตรวจสอบว่าสมาชิกตัวใดตัวหนึ่งมีอยู่ใน Tuple หรือไม่ โดยใช้คีย์เวิร์ด `in` ซึ่งเป็นวิธีการที่ง่ายและมีประสิทธิภาพ

การใช้งาน in

คีย์เวิร์ด `in` ทำหน้าที่ตรวจสอบ "การเป็นสมาชิก" (Membership Test) โดยจะคืนค่าเป็น `True` หากสมาชิกที่ต้องการค้นหา มีอยู่ใน Tuple และจะคืนค่าเป็น `False` หากไม่พบ

รูปแบบการใช้งาน

element in tuple_name

ตัวอย่าง

```
colors = ('red', 'orange', 'blue')

# ตรวจสอบว่า 'yellow' มีอยู่ใน Tuple หรือไม่
print('yellow' in colors)
# ผลลัพธ์: False ('yellow' ไม่มีใน colors)

# ตรวจสอบว่า 'red' มีอยู่ใน Tuple หรือไม่
print('red' in colors)
# ผลลัพธ์: True ('red' มีใน colors)
```

การใช้งาน not in

นอกจากนี้ เรายังสามารถใช้คีย์เวิร์ด `not in` เพื่อตรวจสอบว่าสมาชิก ไม่มี อยู่ใน Tuple หรือไม่

ตัวอย่าง

```
colors = ('red', 'orange', 'blue')

# ตรวจสอบว่า 'yellow' ไม่อยู่ใน Tuple หรือไม่
print('yellow' not in colors)
# ผลลัพธ์: True ('yellow' ไม่อยู่ใน colors)
```

การลบ Tuple

เนื่องจาก Tuple เป็นโครงสร้างข้อมูลที่ไม่สามารถเปลี่ยนแปลงได้ (immutable) จึงไม่สามารถลบสมาชิกแต่ละรายการใน Tuple ได้โดยตรง

อย่างไรก็ตาม สามารถลบ Tuple ทั้งหมด ออกจากหน่วยความจำได้ โดยใช้คำสั่ง del (delete)

การใช้งาน del เพื่อลบ Tuple

การใช้คำสั่ง del ตามด้วยชื่อ Tuple จะทำให้ Tuple นั้นถูกลบออกไป และไม่สามารถเข้าถึงได้อีก

ตัวอย่าง

```
animals = ('dog', 'cat', 'rat')

# ลบ Tuple 'animals'
del animals

# หากเราพยายามเรียกใช้ 'animals' หลังจากลบ จะเกิดข้อผิดพลาด NameError
print(animals)
```

3.6 ชนิดข้อมูลเซต (Set)

Set เป็นโครงสร้างข้อมูลประเภทหนึ่งใน Python ที่ใช้สำหรับจัดเก็บกลุ่มของข้อมูลที่ไม่ซ้ำกัน (unique data)

Sets ถูกออกแบบมาโดยเฉพาะเพื่อจัดการกับข้อมูลที่ไม่ซ้ำกัน หากมีการพยายามเพิ่มสมาชิกที่มีอยู่แล้วเข้าไปใน Set ข้อมูลนั้นจะไม่ถูกเพิ่มซ้ำ และ Set จะคงไว้ซึ่งสมาชิกเดิม

Sets เหมาะสำหรับสถานการณ์ที่ต้องการข้อมูลที่ไม่ซ้ำกัน เช่น การจัดเก็บรหัสนักเรียน (Student IDs) หรือหมายเลขโทรศัพท์

คุณสมบัติที่สำคัญของ Set

1. **ไม่ซ้ำกัน (Unique):** สมาชิกทุกตัวใน Set จะต้องมีเพียงหนึ่งเดียวเท่านั้น หากมีค่าซ้ำกัน Set จะเก็บค่าเหล่านั้นเพียงครั้งเดียว
2. **ไม่มีลำดับ (Unordered):** สมาชิกใน Set จะไม่มีการจัดเรียงตามลำดับเฉพาะ และไม่สามารถเข้าถึงสมาชิกด้วยดัชนี (index) หรือ Slicing ได้
3. **เปลี่ยนแปลงได้ (Mutable):** สามารถเพิ่มหรือลบสมาชิกออกจาก Set ได้
4. **ไม่สามารถเก็บข้อมูลที่เปลี่ยนแปลงได้:** สมาชิกภายใน Set ต้องเป็นชนิดข้อมูลที่ไม่เปลี่ยนแปลง (immutable) เช่น ตัวเลข String หรือ Tuple แต่ไม่สามารถเก็บ List หรือ Dictionary ได้

การสร้าง Set

สร้าง set ด้วย {}

ใน Python สามารถสร้าง Set ได้โดยการใส่สมาชิกทั้งหมดไว้ใน วงเล็บปีกกา {} และคั่นด้วยเครื่องหมาย , (จุลภาค)

Set สามารถมีสมาชิกได้หลายรายการและหลายชนิดข้อมูล เช่น integer, float, string, หรือ Tuple

อย่างไรก็ตาม Set ไม่สามารถมีสมาชิกที่เป็นชนิดข้อมูลที่เปลี่ยนแปลงได้ (mutable) อย่าง List, Set, หรือ Dictionary ได้

ตัวอย่าง

```
# สร้าง Set ของตัวเลข
student_id = {112, 114, 116, 118, 115}

print('Student ID:', student_id)
# ผลลัพธ์: Student ID: {112, 114, 116, 118, 115} (ลำดับอาจแตกต่างกัน)
```

```
# สร้าง Set ของ String
vowel_letters = {'a', 'e', 'i', 'o', 'u'}
print('Vowel Letters:', vowel_letters)
# ผลลัพธ์: Vowel Letters: {'a', 'e', 'i', 'o', 'u'}

# สร้าง Set ที่มีชนิดข้อมูลผสมกัน
mixed_set = {'Hello', 101, -2, 'Bye'}
print('Set of mixed data types:', mixed_set)
# ผลลัพธ์: Set of mixed data types: {'Hello', 101, -2, 'Bye'}
```

ข้อควรทราบ Set ไม่มีลำดับ (Unordered)

เมื่อรันโค้ดข้างต้น อาจสังเกตเห็นว่าลำดับของสมาชิกในผลลัพธ์อาจไม่ตรงกับลำดับที่คุณใส่เข้าไปในโค้ด นี่เป็นเพราะ Set เป็นโครงสร้างข้อมูลที่ไม่มีลำดับ (Unordered) ดังนั้นการเข้าถึงสมาชิกโดยใช้ดัชนีจึงทำไม่ได้

สร้างโดย set ด้วย set() Constructor

นอกจากวงเล็บปีกกา {} ยังสามารถสร้าง Set ได้จาก iterable อื่น ๆ เช่น List หรือ String โดยใช้ set()

ตัวอย่าง

```
# สร้าง Set จาก List
my_list = [1, 2, 3, 2, 1]
unique_set = set(my_list)

print(unique_set)
# ผลลัพธ์: {1, 2, 3}
# สังเกตว่าค่าซ้ำ (2 และ 1) ถูกลบออกไปโดยอัตโนมัติ
```

การสร้าง Empty Set

การสร้าง Set ว่างใน Python นั้นมีความพิเศษเล็กน้อย เนื่องจาก วงเล็บปีกกาเปล่า {} ไม่ได้สร้าง Set ว่าง แต่เป็นการสร้าง Dictionary ว่าง

วิธีการสร้าง Set ว่าง

หากต้องการสร้าง Empty Set (Set ที่ไม่มีสมาชิก) ต้องใช้ฟังก์ชัน `set()` โดยไม่มีอาร์กิวเมนต์ใด ๆ

ตัวอย่าง

```
# สร้าง Empty Set
empty_set = set()
print('Data type of empty_set:', type(empty_set))

# สร้าง Empty Dictionary
empty_dictionary = {}
print('Data type of empty_dictionary:', type(empty_dictionary))
```

การจัดการสมาชิกใน Set

1. การจัดการสมาชิกซ้ำใน Set

Set เป็นโครงสร้างข้อมูลที่เก็บได้เฉพาะข้อมูลที่ไม่ซ้ำกันเท่านั้น หากพยายามเพิ่มสมาชิกซ้ำเข้าไปใน Set, Python จะไม่แสดงข้อผิดพลาด แต่จะเพิกเฉยต่อการเพิ่มสมาชิกนั้น

ตัวอย่าง

```
numbers = {2, 4, 6, 6, 2, 8}
print(numbers)
# ผลลัพธ์: {8, 2, 4, 6} (ลำดับอาจแตกต่างกัน)
```

2. การเพิ่ม อัปเดต และลบสมาชิกใน Set (Sets are Mutable)

Set เป็นโครงสร้างข้อมูลที่สามารถเปลี่ยนแปลงได้ (Mutable) สามารถเพิ่มหรือลบสมาชิกได้ อย่างไรก็ตาม เนื่องจาก Set ไม่มีลำดับ (Unordered) คุณจึงไม่สามารถเข้าถึงหรือแก้ไขสมาชิกโดยใช้ดัชนี (indexing) หรือ Slicing ได้

2.1 การเพิ่มสมาชิกเดี่ยวด้วย `add()`

ใช้เมธอด `add()` เพื่อเพิ่มสมาชิกใหม่เพียงตัวเดียวเข้าไปใน Set

ตัวอย่าง

```
numbers = {21, 34, 54, 12}
print('Initial Set:', numbers)

# เพิ่ม 32 เข้าไปใน Set
numbers.add(32)

print('Updated Set:', numbers)
# ผลลัพธ์: Updated Set: {32, 54, 21, 34, 12} (ลำดับอาจแตกต่างกัน)
```

2.2 การเพิ่มสมาชิกหลายตัวด้วย update()

ใช้เมธอด update() เพื่อเพิ่มสมาชิกจาก iterable อื่น ๆ (เช่น List, Tuple, หรือ Set) เข้าไปใน Set ปัจจุบัน

update() จะเพิ่มเฉพาะสมาชิกที่ไม่ซ้ำกันจาก iterable นั้น ๆ

ตัวอย่าง

```
companies = {'Lacoste', 'Ralph Lauren'}
# tech_companies มี 'apple' ซ้ำกัน
tech_companies = ['apple', 'google', 'apple', 'microsoft']

# ใช้ update() เพื่อเพิ่มสมาชิกจาก List เข้าไปใน Set
companies.update(tech_companies)

print(companies)
# ผลลัพธ์: {'google', 'apple', 'Lacoste', 'Ralph Lauren', 'microsoft'}
```

ในตัวอย่างนี้ สมาชิก apple และ microsoft จาก List ถูกเพิ่มเข้าไปใน Set แต่ apple ที่ซ้ำกันใน List ถูกเพิ่มเพียงครั้งเดียว และลำดับของสมาชิกใน Set จะไม่มีการจัดเรียง

2.3 การลบสมาชิกออกจาก Set

ใน Python เราสามารถลบสมาชิกออกจาก Set ได้หลายวิธี หนึ่งในวิธีที่ง่ายคือการใช้เมธอด discard()

การใช้งาน discard()

เมธอด discard() ใช้สำหรับลบสมาชิกที่ระบุออกจาก Set

- หากสมาชิกที่ระบุมีอยู่ใน Set, สมาชิกนั้นจะถูกลบออก
- หากสมาชิกที่ระบุไม่มีอยู่ใน Set, discard() จะ ไม่ทำอะไรเลยและไม่แสดงข้อผิดพลาด

ตัวอย่าง

```
languages = {'Swift', 'Java', 'Python'}
print('Initial Set:', languages) # ผลลัพธ์: Initial Set: {'Swift', 'Java', 'Python'}

removedValue = languages.discard('Java') # ลบ 'Java' ออกจาก Set

print('Set after discard():', languages) # ผลลัพธ์: Set after discard(): {'Swift', 'Python'}
```

เมธอดอื่น ๆ ในการลบสมาชิก

นอกเหนือจาก discard() ยังมีเมธอดอื่น ๆ ที่ใช้ในการลบสมาชิกออกจาก Set ซึ่งมีความแตกต่างกันเล็กน้อย

1. remove()

- ใช้สำหรับลบสมาชิกที่ระบุ
- ข้อแตกต่าง หากสมาชิกที่ระบุไม่มีอยู่ใน Set, remove() จะแสดงข้อผิดพลาด KeyError

2. pop()

- ใช้สำหรับลบสมาชิกแบบสุ่มออกจาก Set (เนื่องจาก Set ไม่มีลำดับ) และคืนค่าสมาชิกที่ถูกลบ
- หาก Set ว่างเปล่า จะแสดงข้อผิดพลาด KeyError

3. clear()

- ใช้สำหรับลบสมาชิกทั้งหมดใน Set ทำให้ Set ว่างเปล่า

ตัวอย่างการใช้ remove()

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("banana") # ลบ 'banana' ออกจาก Set
print(fruits) # ผลลัพธ์: {'apple', 'cherry'}
fruits.remove("kiwi") # พยายามลบ 'kiwi' ซึ่งไม่มีใน Set ผลลัพธ์: KeyError: 'kiwi'
```

ตัวอย่างการใช้ pop()

```
numbers = {10, 20, 30, 40}
# ลบสมาชิกแบบสุ่ม และเก็บค่าไว้ในตัวแปร
removed_item = numbers.pop()

print("สมาชิกที่ถูกลบ:", removed_item)
print("Set ที่เหลือ:", numbers)
# ผลลัพธ์:
# สมาชิกที่ถูกลบ: (ค่าใดค่าหนึ่ง เช่น 10)
# Set ที่เหลือ: (สมาชิกที่เหลือ)
```

ตัวอย่างการใช้ clear()

```
data_set = {"A", "B", "C", "D"}
print("Set ก่อน clear:", data_set)
# ลบสมาชิกทั้งหมดใน Set
data_set.clear()

print("Set หลัง clear:", data_set)
# ผลลัพธ์: Set หลัง clear: set()
```

การวนซ้ำ (Iteration) ใน Set ด้วย for Loop

สามารถใช้ for loop เพื่อวนซ้ำและเข้าถึงสมาชิกแต่ละตัวใน Set ได้

ตัวอย่าง

```
fruits = {"Apple", "Peach", "Mango"}

# ใช้ for loop เพื่อวนซ้ำผ่านสมาชิกใน Set
for fruit in fruits:
    print(fruit)
```

สิ่งที่ควรจำ เนื่องจาก Set เป็นโครงสร้างข้อมูลที่ไม่มีการลำดับ (Unordered) เมื่อวนซ้ำผ่าน Set ผลลัพธ์ที่ได้อาจจะไม่เรียงตามลำดับที่คุณใส่เข้าไป

การหาความยาวของ Set ด้วย len()

ตัวอย่าง

```
even_numbers = {2, 4, 6, 8}
print('Set:', even_numbers)

# ใช้ len() เพื่อหาจำนวนสมาชิก
print('Total Elements:', len(even_numbers))
# ผลลัพธ์: Total Elements: 4
```

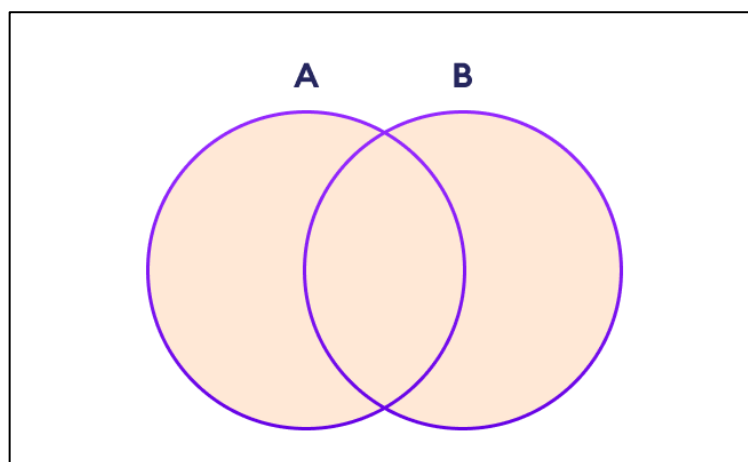
การดำเนินการทางคณิตศาสตร์ของ Sets (Set Operations)

Sets ใน Python ไม่เพียงแต่เก็บข้อมูลที่ไม่ซ้ำกันเท่านั้น แต่ยังสามารถทำการดำเนินการทางคณิตศาสตร์ได้ เช่น Union (ยูเนียน) Intersection (อินเตอร์เซกชัน) Difference (ผลต่าง) และ Symmetric Difference

การดำเนินการเหล่านี้มีประโยชน์อย่างยิ่งในการจัดการกับกลุ่มข้อมูล

ยูเนียนเซต (Union of Sets)

Union คือการรวมสมาชิกทั้งหมดจาก Set A และ Set B เข้าด้วยกัน โดยจะนำสมาชิกที่ไม่ซ้ำกันทั้งหมดมาไว้ใน Set ใหม่



รูป 8 แผนภาพการยูเนียนของเซต ($A \cup B$)

สามารถดำเนินการ Union ได้สองวิธี

- ใช้ตัวดำเนินการ | (Vertical Bar)
- ใช้เมธอด union()

ตัวอย่าง

```
# เซตแรก
A = {1, 3, 5}
# เซตที่สอง
B = {0, 2, 4}
# การทำ Union โดยใช้ | (ตัวดำเนินการ)
print('Union using |: ', A | B) # ผลลัพธ์: Union using |: {0, 1, 2, 3, 4, 5}
# การทำ Union โดยใช้ union() (เมธอด)
print('Union using union(): ', A.union(B)) # ผลลัพธ์: Union using union(): {0, 1, 2, 3, 4, 5}
```

ข้อควรทราบ การดำเนินการ $A | B$ และ $A.union(B)$ ให้ผลลัพธ์เหมือนกันและเทียบเท่ากับ การดำเนินการ $A \cup B$ (A ยูเนียน B) ในคณิตศาสตร์

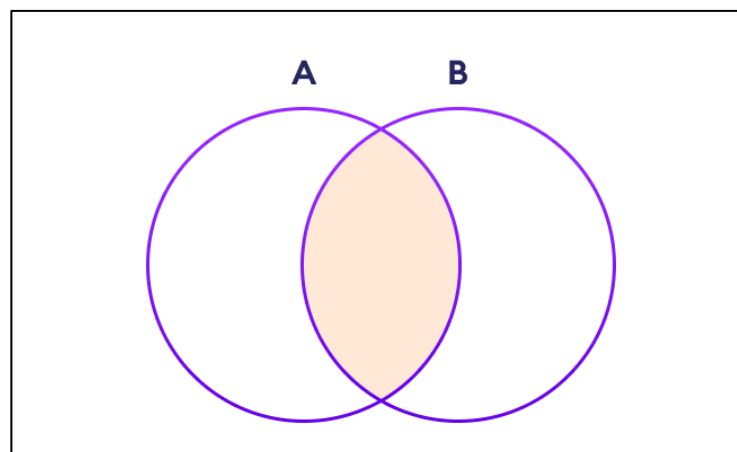
หากมีสมาชิกที่ซ้ำกันใน Sets ทั้งสอง ตัวอย่างเช่น $\{1, 3, 5\}$ และ $\{3, 5, 7\}$ ผลลัพธ์ของ Union จะยังคงเก็บเฉพาะสมาชิกที่ไม่ซ้ำกัน

ตัวอย่าง

```
C = {1, 3, 5}
D = {3, 5, 7}
print(C.union(D)) # ผลลัพธ์: {1, 3, 5, 7}
```

อินเตอร์เซกชันเซต (Intersection of Sets)

Intersection คือการดำเนินการเพื่อหา สมาชิกที่เหมือนกัน (common elements) หรือ สมาชิกที่ปรากฏในทั้งสอง Sets การดำเนินการนี้จะสร้าง Set ใหม่ที่มีเฉพาะสมาชิกที่อยู่ในทั้ง Set A และ Set B



รูป 9 แผนภาพการอินเตอร์เซกชันเซตของเซต ($A \cap B$)

สามารถทำ Intersection ได้สองวิธี

- ใช้ตัวดำเนินการ & (Ampersand)
- ใช้เมธอด intersection()

ตัวอย่าง

```
# เซตแรก
A = {1, 3, 5}

# เซตที่สอง
B = {1, 2, 3}

# 1. การทำ Intersection โดยใช้ & (ตัวดำเนินการ)
print('Intersection using &:', A & B) # ผลลัพธ์: Intersection using &: {1, 3}

# 2. การทำ Intersection โดยใช้ intersection() (เมธอด)
print('Intersection using intersection():', A.intersection(B))

# ผลลัพธ์: Intersection using intersection(): {1, 3}
```

ข้อควรทราบ ทั้ง $A \& B$ และ $A.intersection(B)$ ให้ผลลัพธ์เดียวกัน การดำเนินการนี้เทียบเท่ากับ $A \cap B$ (A อินเตอร์เซกชัน B) ในคณิตศาสตร์

Intersection มีประโยชน์ในการค้นหาสมาชิกที่ซ้ำกันระหว่างกลุ่มข้อมูล

ตัวอย่าง

```
# รายชื่อนักเรียนในชมรมดนตรี
music_club = {'Alice', 'Bob', 'Charlie', 'David'}

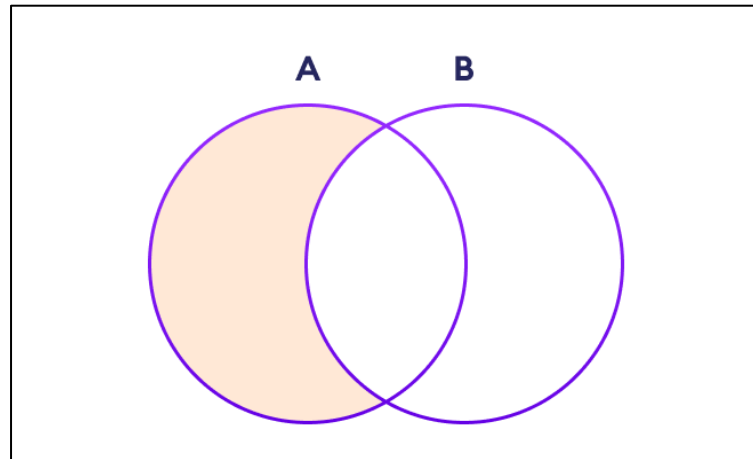
# รายชื่อนักเรียนในชมรมกีฬา
sports_club = {'Bob', 'David', 'Eve', 'Frank'}

# หาว่าใครอยู่ทั้งสองชมรม
common_members = music_club.intersection(sports_club)

print(common_members) # ผลลัพธ์: {'Bob', 'David'}
```

ผลต่างของเซต (Difference of Sets)

Set Difference คือการดำเนินการเพื่อหาสมาชิกที่อยู่ใน Set แรก แต่ไม่อยู่ใน Set ที่สอง การดำเนินการนี้จะสร้าง Set ใหม่ที่มีเฉพาะสมาชิกที่แตกต่างกันระหว่าง Set A และ Set B



รูป 10 แผนภาพผลต่างของเซต ($A - B$)

สามารถทำ Difference ได้สองวิธี

- ใช้ตัวดำเนินการ - (Hyphen)
- ใช้เมธอด difference()

ตัวอย่าง

```
A = {2, 3, 5} # เซตแรก
B = {1, 2, 6} # เซตที่สอง

# การทำ Difference โดยใช้ - (ตัวดำเนินการ)
print('Difference using -:', A - B) # ผลลัพธ์: Difference using -: {3, 5} สมาชิก 3 และ 5 อยู่ใน A แต่ไม่อยู่ใน B

# การทำ Difference โดยใช้ difference() (เมธอด)
print('Difference using difference():', A.difference(B)) # ผลลัพธ์: Difference using difference(): {3, 5}
```

ข้อควรทราบ ทั้ง $A - B$ และ $A.difference(B)$ ให้ผลลัพธ์เดียวกัน การดำเนินการนี้เทียบเท่ากับ $A - B$ ในคณิตศาสตร์

ลำดับมีความสำคัญ $A - B$ จะได้ผลลัพธ์ที่แตกต่างจาก $B - A$

ตัวอย่าง

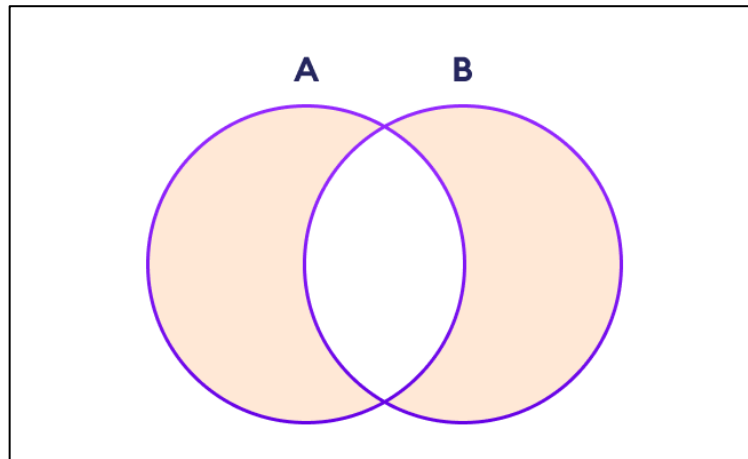
```
A = {2, 3, 5} # เซตแรก
B = {1, 2, 6} # เซตที่สอง

print(B - A) # ผลลัพธ์: {1, 6} สมาชิก 1 และ 6 อยู่ใน B แต่ไม่อยู่ใน A
```

ผลต่างสมมาตรของเซต (Symmetric Difference of Sets)

Symmetric Difference คือการดำเนินการเพื่อหาสมาชิกทั้งหมดที่อยู่ใน Set A หรือ Set B แต่ไม่ใช่สมาชิกที่อยู่ในทั้งสอง Set พร้อมกัน

เป็นการรวมสมาชิกทั้งหมด ยกเว้นสมาชิกที่ซ้ำกัน (common elements) ที่อยู่ใน Intersection



รูป 11 แผนภาพผลต่างสมมาตรของเซต ($A - B$)

สามารถทำ Symmetric Difference ได้สองวิธี

- ใช้ตัวดำเนินการ $\wedge$ (Caret)
- ใช้เมธอด `symmetric_difference()`

ตัวอย่าง

```
A = {2, 3, 5} # เซตแรก
B = {1, 2, 6} # เซตที่สอง

# สมาชิกที่ซ้ำกัน (Intersection) คือ {2}   สมาชิกที่ไม่ซ้ำกันใน A คือ {3, 5}
# สมาชิกที่ไม่ซ้ำกันใน B คือ {1, 6}

# การทำ Symmetric Difference โดยใช้  $\wedge$  (ตัวดำเนินการ)
print('using  $\wedge$ :', A  $\wedge$  B) # ผลลัพธ์: using  $\wedge$ : {1, 3, 5, 6}

# การทำ Symmetric Difference โดยใช้ symmetric_difference() (เมธอด)
print('using symmetric_difference():', A.symmetric_difference(B))
# ผลลัพธ์: using symmetric_difference(): {1, 3, 5, 6}
```

ข้อควรทราบ ทั้ง $A \wedge B$ และ `A.symmetric_difference(B)` ให้ผลลัพธ์เดียวกัน

การดำเนินการนี้เทียบเท่ากับการรวม (Union) ของ Set A และ B ลบด้วยการหาจุดร่วม (Intersection) $(A \cup B) - (A \cap B)$

สรุปการดำเนินการ Set

การดำเนินการ	ตัวดำเนินการ	เมธอด	ความหมาย
Union		<code>union()</code>	สมาชิกทั้งหมดใน A และ B
Intersection	&	<code>intersection()</code>	สมาชิกที่อยู่ในทั้ง A และ B
Difference	-	<code>difference()</code>	สมาชิกที่อยู่ใน A แต่ไม่อยู่ใน B
Symmetric Difference	$\wedge$	<code>symmetric_difference()</code>	สมาชิกที่อยู่ใน A หรือ B แต่ไม่อยู่ในทั้งคู่

3.7 ชนิดข้อมูลดิกชันนารี (Dictionary)

Dictionary (พจนานุกรม) เป็นโครงสร้างข้อมูลประเภทหนึ่งใน Python ที่ใช้สำหรับจัดเก็บข้อมูลในรูปแบบของคู่ Key-Value (กุญแจ-ค่า)

Dictionary มีลักษณะคล้ายกับ List และ Tuple ตรงที่เป็นการจัดเก็บข้อมูลหลายรายการ แต่แตกต่างกันตรงที่ Dictionary ไม่ได้ใช้ลำดับดัชนี (index) แบบตัวเลขในการเข้าถึงข้อมูล แต่ใช้ Key แทน

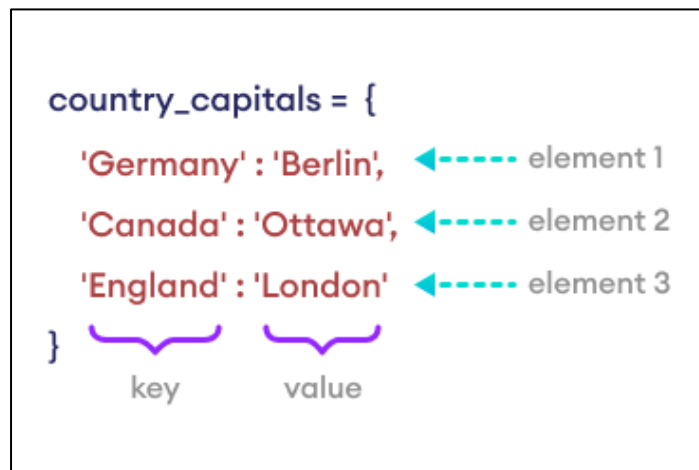
คุณสมบัติหลักของ Dictionary

- Key-Value Pairs ข้อมูลจะถูกจัดเก็บเป็นคู่ โดยแต่ละคู่ประกอบด้วย Key และ Value
- มีลำดับ (Ordered) ตั้งแต่ Python เวอร์ชัน 3.7 เป็นต้นไป Dictionary จะจดจำลำดับการเพิ่ม Key-Value pairs เข้าไปใน Dictionary และรักษาระดับนั้นไว้เมื่อมีการเข้าถึงหรือวนซ้ำ (iterate)
- เปลี่ยนแปลงได้ (Mutable) สามารถเพิ่ม แก้ไข หรือลบ Key-Value pairs ได้
- Key ต้องไม่ซ้ำกัน Key แต่ละตัวใน Dictionary จะต้องไม่ซ้ำกัน
- Key ต้องเป็น Immutable Key ต้องเป็นข้อมูลที่ไม่สามารถเปลี่ยนแปลงได้ (hashable) เช่น String, Integer, Float, หรือ Tuple แต่ไม่สามารถใช้ List หรือ Set เป็น Key ได้

การสร้าง Dictionary

สร้าง Dictionary โดย key: value

สร้าง Dictionary โดยการใส่คู่ key: value ไว้ใน วงเล็บปีกกา {} และคั่นด้วยเครื่องหมาย ,



รูป 12 การสร้าง Dictionary โดย key: value

ตัวอย่าง สร้าง Dictionary ที่เก็บชื่อประเทศและเมืองหลวง

```
country_capitals = {
    "Germany": "Berlin", # "Germany" คือ Key, "Berlin" คือ Value
    "Canada": "Ottawa",
    "England": "London"
}
print(country_capitals) # ผลลัพธ์: {'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}
```

ในตัวอย่างนี้ country_capitals มี 3 รายการ แต่ละรายการเป็นคู่ Key-Value

การสร้าง Dictionary ด้วย dict()

สามารถใช้ dict() constructor เพื่อสร้าง Dictionary ได้

ตัวอย่าง สร้าง Dictionary โดยใช้ dict() จากรายการ (list of tuples) ของ Key-Value pairs

```
phone_book = dict([
    ('John', 1234567),
    ('Jane', 9876543),
    ('Mike', 1122334)
])
print(phone_book) # ผลลัพธ์: {'John': 1234567, 'Jane': 9876543, 'Mike': 1122334}
```

การสร้าง Dictionary ที่ถูกต้องและไม่ถูกต้อง

Dictionary มีข้อกำหนดที่สำคัญเกี่ยวกับ Key คือ Key จะต้องเป็นข้อมูลที่ไม่สามารถเปลี่ยนแปลงได้ (Immutable) เท่านั้น

ตัวอย่าง Dictionary ที่ถูกต้อง (Valid)

สามารถใช้ข้อมูลประเภท Integer, Tuple, และ String เป็น Key ได้ เนื่องจากเป็นข้อมูลประเภท Immutable

ตัวอย่าง

```
# valid dictionary: ใช้ integer เป็น key
my_dict = {1: "one", 2: "two", 3: "three"}
```

ตัวอย่าง

```
# valid dictionary: ใช้ tuple เป็น key
my_dict = {(1, 2): "one two", 3: "three"}
```

ตัวอย่าง

```
# valid dictionary: ใช้ string เป็น key
my_dict = {"apple": 1, "banana": 2}
```

ตัวอย่าง Dictionary ที่ไม่ถูกต้อง (Invalid)

ไม่สามารถใช้ข้อมูลประเภท Mutable อย่าง List, Set, หรือ Dictionary เป็น Key ได้

ตัวอย่าง

```
# invalid dictionary: การใช้ list เป็น key จะเกิดข้อผิดพลาด TypeError
my_dict = {1: "Hello", [1, 2]: "Hello Hi"}
```

หากรันโค้ดนี้ จะได้รับข้อผิดพลาด: `TypeError: unhashable type: 'list'`

ข้อควรทราบ Values สามารถเป็น Mutable ได้

แม้ว่า Key จะต้องเป็น Immutable แต่ Values (ค่า) ใน Dictionary สามารถเป็นข้อมูลประเภทใดก็ได้ รวมถึงข้อมูลประเภท Mutable เช่น List หรือ Dictionary

Dictionary Keys ต้องไม่ซ้ำกัน (Unique)

Key ของ Dictionary จะต้องไม่ซ้ำกัน (Unique) ซึ่งเป็นข้อกำหนดสำคัญของการใช้งาน Dictionary หากพยายามกำหนด Key ที่ซ้ำกันใน Dictionary ค่าใหม่จะเขียนทับ (overwrite) ค่าเดิม

การทำงานเมื่อมี Key ซ้ำกัน

เมื่อ Python พบ Key ที่ซ้ำกันใน Dictionary ระบบจะใช้ค่าที่กำหนดล่าสุดสำหรับ Key นั้นเสมอ และจะไม่มีอาการแจ้งเตือนข้อผิดพลาดใด ๆ

ตัวอย่าง

```
hogwarts_houses = {
    "Harry Potter": "Gryffindor",
    "Hermione Granger": "Gryffindor",
    "Ron Weasley": "Gryffindor",
    # มี Key 'Harry Potter' ซ้ำกัน
    "Harry Potter": "Slytherin"
}

print(hogwarts_houses)
# ผลลัพธ์: {'Harry Potter': 'Slytherin', 'Hermione Granger': 'Gryffindor', 'Ron Weasley': 'Gryffindor'}
```

ในตัวอย่างนี้

1. Key **"Harry Potter"** ถูกกำหนดค่าแรกเป็น **"Gryffindor"**
2. เมื่อ Python พบ Key **"Harry Potter"** อีกครั้งพร้อมกับค่า **"Slytherin"** ค่าใหม่นี้จะเขียนทับค่าเดิม

ผลลัพธ์ที่ได้คือ Dictionary จะเก็บ Key **"Harry Potter"** ไว้เพียงครั้งเดียว โดยมีค่าเป็น **"Slytherin"**

เหตุผลที่ Key ต้อง Unique

Dictionary ถูกออกแบบมาเพื่อการค้นหาข้อมูลที่รวดเร็วและมีประสิทธิภาพ โดยใช้ Key เป็นตัวระบุที่ไม่ซ้ำกัน (unique identifier) หาก Key สามารถซ้ำกันได้ ระบบจะไม่สามารถระบุได้ว่า

ต้องการเข้าถึงค่าใด ทำให้เกิดความสับสนและไม่สามารถทำงานได้ตามวัตถุประสงค์ของ Dictionary

การเข้าถึงสมาชิกใน Dictionary

การเข้าถึงค่า (Value) ใน Dictionary ทำได้โดยใช้ Key

เนื่องจาก Dictionary จัดเก็บข้อมูลในรูปแบบ Key-Value pair จึงไม่สามารถใช้ดัชนีตัวเลขเหมือน List หรือ Tuple ได้

1. การเข้าถึงโดยใช้ []

วิธีที่พบบ่อยที่สุดในการเข้าถึง Value คือการใส่ Key ไว้ในวงเล็บเหลี่ยม [] ต่อท้ายชื่อ Dictionary

รูปแบบ: `dictionary_name[key]`

ตัวอย่าง

```
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
    "England": "London"
}
# เข้าถึงค่าของ Key "Germany"
print(country_capitals["Germany"]) # Output: Berlin
# เข้าถึงค่าของ Key "England"
print(country_capitals["England"]) # Output: London
```

ข้อควรระวัง หากพยายามเข้าถึง Key ที่ไม่มีอยู่ใน Dictionary โดยใช้วิธีนี้ Python จะแสดงข้อผิดพลาด `KeyError`

ตัวอย่าง

```
print(country_capitals["USA"])
# จะเกิด KeyErrors["USA"]
```

2. การเข้าถึงโดยใช้เมธอด get()

สามารถใช้เมธอด get() เพื่อเข้าถึงสมาชิกได้ ซึ่งเป็นวิธีที่ปลอดภัยกว่า หาก Key ที่ระบุไม่มีอยู่ใน Dictionary เมธอดนี้จะคืนค่าเป็น None แทนที่จะเกิดข้อผิดพลาด

รูปแบบ: `dictionary_name.get(key, default_value)`

ตัวอย่าง

```
print(country_capitals.get("Canada")) # Output: Ottawa
# หาก Key 'USA' ไม่มีอยู่ใน Dictionary จะคืนค่า None (ค่าเริ่มต้น)
print(country_capitals.get("USA"))    # Output: None
# คุณสามารถกำหนดค่าเริ่มต้นได้ หากไม่พบ Key
print(country_capitals.get("USA", "Not Found")) # Output: Not Found
```

การใช้ get() เหมาะกับการเข้าถึงข้อมูลที่คุณไม่แน่ใจว่า Key นั้นมีอยู่หรือไม่

การจัดการสมาชิกใน Python Dictionary

สามารถเพิ่ม ลบ หรือแก้ไขสมาชิกใน Dictionary ได้ เนื่องจาก Dictionary เป็นโครงสร้างข้อมูลที่เปลี่ยนแปลงได้ (Mutable)

1. การเพิ่มและแก้ไขสมาชิกใน Dictionary (Add and Update)

การเพิ่ม Key-Value pair

สามารถเพิ่ม Key-Value pair ใหม่เข้าไปใน Dictionary ได้โดยการกำหนดค่าให้กับ Key ใหม่ที่เราต้องการ

ตัวอย่าง

```
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
}
country_capitals["Italy"] = "Rome" # เพิ่ม 'Italy' เป็น Key และ 'Rome' เป็น Value
print(country_capitals) # ผลลัพธ์: {'Germany': 'Berlin', 'Canada': 'Ottawa', 'Italy': 'Rome'}
```

หมายเหตุ หาก Key ที่คุณใช้ในการเพิ่มมีอยู่แล้ว การดำเนินการนี้จะกลายเป็นการ แก้ไข (Update) ค่าของ Key นั้นแทน

การใช้ update()

เมธอด update() ใช้สำหรับการเพิ่มหรืออัปเดต Dictionary โดยใช้ Key-Value pairs จาก Dictionary อื่น หรือจาก iterable ที่เป็นคู่ เช่น List ของ Tuples

ตัวอย่าง

```
my_dict = {'a': 1, 'b': 2}
new_data1 = {'b': 20, 'c': 3}
# เพิ่ม 'c': 3 และอัปเดต 'b' เป็น 20
my_dict.update(new_data1)
print(my_dict)
# ผลลัพธ์: {'a': 1, 'b': 20, 'c': 3}
```

ตัวอย่าง

```
# Dictionary เดิม: เก็บชื่อนักเรียนและคะแนน
student_scores = {
    'Alice': 85,
    'Bob': 78,
    'Charlie': 92
}

# List ของ Tuples ที่เก็บข้อมูลวิชาใหม่
# รูปแบบ: [(key, value), (key, value), ...]
new_scores_list = [
    ('Alice', 90), # Key 'Alice' มีอยู่แล้ว, จะถูกอัปเดต
    ('David', 75), # Key 'David' ยังไม่มี, จะถูกเพิ่ม
    ('Bob', 80)   # Key 'Bob' มีอยู่แล้ว, จะถูกอัปเดต
]

# ใช้ update() เพื่อเพิ่มและอัปเดตข้อมูลจาก List ของ Tuples
student_scores.update(new_scores_list)

print(student_scores)
```

2. การลบสมาชิกออกจาก Dictionary

สามารถลบ Key-Value pair ออกจาก Dictionary ได้หลายวิธี

การลบด้วย del Statement

ใช้คำสั่ง del ตามด้วยชื่อ Dictionary และ Key ที่ต้องการลบ

ตัวอย่าง

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
  
# ลบรายการที่มี Key เป็น "Germany"  
del country_capitals["Germany"]  
  
print(country_capitals)  
# ผลลัพธ์: {'Canada': 'Ottawa'}
```

การลบด้วยเมธอด pop()

เมธอด pop() ใช้สำหรับลบสมาชิกที่ระบุ Key และ คืนค่า (return) Value ของ Key นั้นกลับมา

ตัวอย่าง

```
countries = {"USA": "Washington DC", "Japan": "Tokyo"}  
  
# ลบ 'USA' และเก็บค่า 'Washington DC' ไว้ในตัวแปร  
removed_capital = countries.pop("USA")  
  
print(f"Removed Capital: {removed_capital}") # Output: Removed Capital: Washington DC  
print(countries) # Output: {'Japan': 'Tokyo'}
```

การลบสมาชิกทั้งหมดด้วย clear()

เมธอด clear() ใช้สำหรับลบสมาชิกทั้งหมดใน Dictionary ทำให้ Dictionary ว่างเปล่า

ตัวอย่าง

```
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
}
# ลบสมาชิกทั้งหมด
country_capitals.clear()

print(country_capitals) # ผลลัพธ์: {} (Empty Dictionary)
```

การวนซ้ำ (Iterate) ใน Dictionary

การวนซ้ำ (iteration) ใน Dictionary ช่วยให้เราสามารถเข้าถึง Key หรือ Value ของแต่ละคู่ข้อมูลได้

ตั้งแต่ Python เวอร์ชัน 3.7 เป็นต้นไป Dictionary จะรักษาลำดับการเพิ่ม Key-Value pairs ดังนั้นเมื่อวนซ้ำ จะได้ข้อมูลตามลำดับที่เพิ่มเข้าไป

1. การวนซ้ำเพื่อเข้าถึง Keys

ใช้ for loop กับ Dictionary โดยตรง ลูปจะวนซ้ำผ่าน Keys ของ Dictionary

ตัวอย่าง

```
country_capitals = {
    "United States": "Washington D.C.",
    "Italy": "Rome"
}
# วนซ้ำเพื่อพิมพ์เฉพาะ Key (ชื่อประเทศ)
for country in country_capitals:
    print(country)
# ผลลัพธ์:
# United States
# Italy
```


2. การวนซ้ำเพื่อเข้าถึง Values

หากต้องการเข้าถึง Values (เช่น เมืองหลวง) สามารถใช้ Key ที่ได้จากการวนซ้ำเพื่อดึง Value ที่เกี่ยวข้อง

ตัวอย่าง

```
country_capitals = {
    "United States": "Washington D.C.",
    "Italy": "Rome"
}
# วนซ้ำเพื่อพิมพ์เฉพาะ Value (เมืองหลวง)
for country in country_capitals:
    capital = country_capitals[country]
    print(capital)
# ผลลัพธ์:
# Washington D.C.
# Rome
```

3. การวนซ้ำด้วยเมธอด items(), keys(), และ values()

Python มีเมธอดที่ทำให้การวนซ้ำชัดเจนและง่ายขึ้น

- items(): วนซ้ำเพื่อรับทั้ง Key และ Value ในรูปแบบ Tuple
- keys(): วนซ้ำเพื่อรับเฉพาะ Keys (เหมือนกับการวนลูปโดยตรง)
- values(): วนซ้ำเพื่อรับเฉพาะ Values

ตัวอย่างวนซ้ำผ่าน Key และ Value โดยใช้ .items()

```
country_capitals = {
    "United States": "Washington D.C.",
    "Italy": "Rome"
}
# วนซ้ำผ่าน Key และ Value โดยใช้ .items()
for country, capital in country_capitals.items():
    print(f"The capital of {country} is {capital}.")
# ผลลัพธ์:
# The capital of United States is Washington D.C..
# The capital of Italy is Rome.
```

ตัวอย่างวนซ้ำโดยใช้ .keys()

```
country_capitals = {  
    "United States": "Washington D.C.",  
    "Italy": "Rome",  
    "Japan": "Tokyo"  
}  
# วนซ้ำโดยใช้ .keys()  
for country in country_capitals.keys():  
    print(country)  
# ผลลัพธ์:  
# United States  
# Italy  
# Japan
```

ตัวอย่างวนซ้ำโดยใช้ .values()

```
country_capitals = {  
    "United States": "Washington D.C.",  
    "Italy": "Rome",  
    "Japan": "Tokyo"  
}  
# วนซ้ำโดยใช้ .values()  
for capital in country_capitals.values():  
    print(capital)  
# ผลลัพธ์:  
# Washington D.C.  
# Rome  
# Tokyo
```

การตรวจสอบการเป็นสมาชิกใน Dictionary (Membership Test)

สามารถตรวจสอบว่า Key ใด ๆ มีอยู่ใน Dictionary หรือไม่ โดยใช้ตัวดำเนินการ in และ not in

การใช้งาน in และ not in

การตรวจสอบการเป็นสมาชิกใน Dictionary จะดำเนินการกับ Keys เท่านั้น ไม่ใช่ Values:

- in: คืนค่า True หาก Key ที่ระบุมีอยู่ใน Dictionary
- not in: คืนค่า True หาก Key ที่ระบุไม่มีอยู่ใน Dictionary

ตัวอย่าง

```
file_types = {
    ".txt": "Text File",
    ".pdf": "PDF Document",
    ".jpg": "JPEG Image",
}
# ตรวจสอบการเป็นสมาชิกโดยใช้ 'in'
print(".pdf" in file_types)    # ผลลัพธ์: True (Key '.pdf' มีอยู่)
print(".mp3" in file_types)    # ผลลัพธ์: False (Key '.mp3' ไม่มีอยู่)
# ตรวจสอบการเป็นสมาชิกโดยใช้ 'not in'
print(".mp3" not in file_types) # ผลลัพธ์: True (Key '.mp3' ไม่มีอยู่)
```

ข้อควรจำ การตรวจสอบเฉพาะ Keys

ตัวดำเนินการ in และ not in จะค้นหาเฉพาะใน Keys ของ Dictionary เท่านั้น

หากต้องการตรวจสอบว่า Value ใด ๆ มีอยู่ใน Dictionary หรือไม่ จะต้องใช้เมธอด .values() ก่อน

ตัวอย่าง

```
file_types = {
    ".txt": "Text File",
    ".pdf": "PDF Document",
    ".jpg": "JPEG Image",
}
# ตรวจสอบว่า "JPEG Image" มีอยู่ใน Values หรือไม่
print("JPEG Image" in file_types.values()) # ผลลัพธ์: True
```